

Visual odometry for autonomous vehicles in a constrained environment

Report submitted to GITAM (Deemed to be University) as a partial
fulfillment of the requirements for the award of the Degree of Bachelor
of Technology in (ECE)

Team Members

Thippeswamy ks	BU22EECE0100422
Sai bharath M	BU22EECE0100467
M Gopichand	BU22EECE0100474



GITAM
DEEMED TO BE UNIVERSITY

DEPARTMENT OF ELECTRICAL, ELECTRONICS AND COMMUNICATION
ENGINEERING
GITAM SCHOOL OF TECHNOLOGY
GITAM (DEEMED TO BE UNIVERSITY)
BENGALURU -561203
NOV 2025

DECLARATION

I/We declare that the project work contained in this report is original and it has been done by me under the guidance of my project guide.

Name:Thippeswamy ks, Sai bharath M, M Gopichand

Date: 26th September 2025

Signature of the student: Thippeswamy Ks

**Department of Electrical, Electronics and Communication
Engineering GITAM School of Technology, Bengaluru-561203**



CERTIFICATE

This is to certify that **Thippeswamy ks** bearing **BU22EECE0100422** has satisfactorily completed Mini Project Entitled in partial fulfillment of the requirements as prescribed by University for VIIth semester, Bachelor of Technology in “Electrical, Electronics and Communication Engineering” and submitted this report during the academic year 2025-2026.

[Signature of the Guide]

[Signature of HOD]

Table of content

DECLARATION	2
Table of content	4
1. Fundamentals of Visual Odometry	6
1.1. Introduction to Visual Odometry	6
1.2. What is Visual Odometry?	6
1.2.1. Definition and Core Concept	6
1.3. Difference from SLAM and Visual SLAM	8
1.4. Importance in Robotics and Computer Vision	9
1.4.1. Importance in Robotics	9
1.4.2. Importance in Computer Vision	10
1.5. Types of Visual Odometry by Sensor	10
1.5.1. Monocular Visual Odometry	10
1.5.2. Stereo Visual Odometry	11
1.5.3. RGB-D Visual Odometry	12
1.6. Data Types and Sensor Fusion in Visual Odometry	14
1.6.1. Key Data Types	14
1.6.2. Sensor Fusion Strategies	16
2. State-of-the-Art SLAM Techniques of Visual Odometry	18
2.1. ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras	18
2.1.1. Introduction and System Architecture	18
2.1.2. Key Innovation: Unification of Stereo and RGB-D Sensors	19
2.1.3. System Details: Multi-View Point Triangulation	20
2.1.4. Experimental Results and Conclusion	21
2.2. DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras	21
2.2.1. Introduction and Core Concept	21
2.2.2. Key Innovation: Recurrent Updates and Differentiable DBA	22
2.2.3. Training, Generalization, and System Details	22
2.2.4. Experimental Results and Conclusion	23
3. Industry Problems and Applications of Visual Odometry	23
3.1. vSLAM Application Areas	23
3.2. Case Study: Virya Autonomous Technologies	24
3.2.1. What Virya Autonomous Technologies Does	24
3.2.2. Primary Products and Solutions	24
3.2.3. Core Technology and Relation to vSLAM	25
3.3. Localization and Path Planning in Industry	25
3.3.1. Overview of the Core Problem	25
3.3.2. Common Industrial Approach (e.g., Virya)	25
3.4. Future Trends and Challenges in Industrial Applications	26
3.4.1. Future Trends	26
4. Paper Survey: Sensors, Algorithms, and Dataset	26
4.1. Sensors for Visual Odometry	26
4.1.1. Monocular Cameras	26
4.1.2. Stereo Cameras	29
4.1.3. RGB-D Cameras	30

4.1.4. Lidar (Brief Mention in Context of Fusion)	32
4.2. Sensor Calibration and Synchronization	33
4.2.1. Methods for Camera Calibration (Intrinsic)	33
4.2.2. IMU Calibration (Intrinsic)	34
4.2.3. Multi-Sensor Extrinsic Calibration	35
4.2.4. Time Synchronization Techniques	36
5. Autonomous Buggy vehicle	37
5.1. Autonomous Buggy Platform Overview	37
5.2. Vehicle Dimensions and Mechanical Specifications	37
5.3. Sensor and Perception Stack: Detailed Configuration	38
5.3.1. Primary Localization and Visual-Inertial (V-I) Sensors	38
5.4.1. External Sensor Placement Overview	40
5.5. Actuation, Control, and Computational Environment	42
5.5.1. Actuation and Power Subsystems	42
5.5.2. Current Navigation Strategy: The LiDAR-SLAM Workflow	42
5.5.3. Visual Data Collection for Development	42
5.5.4. Software Environment	43
6. Implementations and Simulations on Visual Odometry	43
6.1 DROID-SLAM Overview	44
6.1.1 Architecture	44
6.1.2 Co-Visibility Graph	45
6.1.3 Feature Extraction	45
6.1.4 Correlation Pyramid	45
6.1.5 Update Operator	46
6.1.7 Correlation Lookup	47
6.1.8 ConvGRU Layer	47
6.1.9 Dense Bundle Adjustment (DBA) Layer	48
6.2 Lunar Rover Simulation and Navigation Stack	49
6.2.1 Overview 6.2.2 System Architecture Flow	49
6.2.2 System Architecture Flow	49
6.2.3 Simulation Environment and Robot Model	50
6.2.4 Sensor Configuration	50
6.2.5 2D SLAM and Mapping	51
6.2.6 2D Localization and Navigation (Nav2)	51
Accurate Robot Navigation	51
6.2.7 Limitations and Future Work: 3D VSLAM for Lunar Terrain	51
6.3.1 Overview	52
6.3.2 Physical Testbed and Platform Model	52
6.3.3 Sensor Configuration and Data Synchronization	53
6.3.4 Ground-Truth Data Generation (LiDAR-SLAM)	53
6.3.5 Future Work: VSLAM Validation Workflow and System Integration	53
6.3.5.1 Offline Processing and Benchmarking	53
6.3.5.2 Real-Time Deployment and Optimization	54
6.3.5.3 Robust Sensor Fusion (VSLAM + LiDAR-SLAM)	54
6.3.5.4 Advanced Visual Perception and Navigation	54
7. Conclusion	54

7.1. Summary of Key Findings and Contributions of the Report	54
7.2. Future Work and Research Outlook	55
1.1 Overview of the Problem Statement	58
1.2 Objectives and Goals	58
1.2.1 Objective	58
1.2.2 Goals	59
2.2 Foundations in Feature-Based SLAM	60
2.3 Evolution and Application of Visual Odometry	61
2.4 Emergence of Deep Learning in Visual Localization	61
Chapter 4: Methodology	62
4.1 Description of the Approach	62
4.2 Tools and Techniques Utilized	63

1. Fundamentals of Visual Odometry

1.1. Introduction to Visual Odometry

Visual Odometry (VO) stands as a critical technology within the fields of computer vision and robotics, enabling the estimation of an agent's motion purely through the analysis of sequential images captured by its onboard cameras. It serves as a digital counterpart to traditional odometry, which relies on mechanical means like wheel encoders to measure distance travelled. Instead of counting wheel rotations, VO meticulously tracks the apparent motion of visual elements within the camera's field of view to incrementally deduce the camera's path through an environment. This capability is foundational for autonomous navigation, augmented reality, and various applications requiring spatial awareness without reliance on external positioning systems like GPS..

1.2. What is Visual Odometry?

1.2.1. Definition and Core Concept

At its heart, Visual Odometry is the process of estimating the egomotion—specifically, the change in position and orientation over time—of a moving agent (e.g., robot, vehicle, human) using only the input from one or more cameras attached to it. It tackles the challenge of determining "Where am I?" relative to a starting point by observing "How did the view change?". The underlying principles are rooted in multi-view geometry, which relates the 2D projections of 3D points seen from different viewpoints.

A typical VO pipeline, often functioning as the real-time "frontend" of a more comprehensive Simultaneous Localization and Mapping (SLAM) system, involves a sequence of computational steps executed for each new incoming camera frame:

- **Image Acquisition:** Capturing a continuous stream of images from the camera(s). The frame rate and image quality significantly impact VO performance.
- **Feature Detection (for Feature-Based VO):** Identifying salient and stable points or regions in the image that can be reliably detected and tracked across multiple frames. These features act as anchors in the visual world. Common choices include:
 - **Corner Detectors:** Algorithms like FAST, Harris, or Shi-Tomasi excel at finding points with high intensity gradients in multiple directions, which are generally stable under viewpoint changes.
 - **Blob Detectors:** Methods like SIFT (Scale-Invariant Feature Transform) or SURF (Speeded Up Robust Features) detect interest points based on blob-like structures at characteristic scales, providing scale invariance.
 - **ORB Features:** A popular choice in real-time systems like ORB-SLAM, combining the FAST keypoint detector with the BRIEF descriptor, offering rotation invariance and good speed.
- **Alternatively, Direct Methods** bypass explicit feature detection. They operate directly on pixel intensity values, either densely (using all pixels), semi-densely (using pixels with significant intensity gradients), or sparsely (using selected pixels), aiming to minimize photometric errors between frames.
- **Feature Matching/Tracking or Direct Alignment:** Establishing correspondences between features detected in consecutive frames (feature-based) or finding the transformation that best aligns pixel intensities (direct).
 - **Feature Matching:** Involves comparing descriptors (e.g., SIFT, SURF, ORB descriptors) associated with detected features to find likely pairs across frames. Techniques like nearest-neighbor search followed by ratio tests are common. Outlier rejection using algorithms like RANSAC (Random Sample Consensus) is crucial to discard incorrect matches.

- **Feature Tracking:** Algorithms like the Kanade-Lucas-Tomasi (KLT) tracker directly track the motion of feature patches between frames using optical flow principles, assuming brightness constancy and small displacements.
- **Direct Alignment:** Involves optimizing the relative camera pose to minimize the photometric difference (sum of squared differences or other robust metrics) between the current frame and a warped version of the previous frame(s), based on the estimated pose and potentially a depth map.
- **Motion Estimation:** Recovering the camera's 3D motion (rotation R and translation t) that best explains the observed 2D motion of features or pixel intensities. The geometric principles differ based on the available information:
 - **2D-to-2D (Epipolar Geometry):** If only 2D correspondences between two frames are known (typical in monocular VO initialization), the Essential Matrix (for calibrated cameras) or Fundamental Matrix (for uncalibrated cameras) is estimated using algorithms like the 8-point or 5-point algorithm within RANSAC. This matrix is then decomposed to yield possible rotation and translation solutions (up to scale for monocular).
 - **3D-to-2D (Perspective-n-Point - PnP):** If correspondences between known 3D map points and their 2D projections in the current frame are available (common during tracking after initialization), algorithms like PnP (e.g., EPnP, DLS) solve for the camera's absolute pose relative to the map points. This requires at least 3 correspondences.
 - **3D-to-3D (Point Cloud Alignment):** If 3D point clouds are available for consecutive frames (e.g., from stereo or RGB-D VO), algorithms like Iterative Closest Point (ICP) can directly align the clouds to find the relative motion.
- **Bundle Adjustment (BA):** Often used for refinement, BA is a joint non-linear optimization that minimizes the reprojection error (the difference between observed 2D feature locations and the predicted projections of estimated 3D points based on estimated camera poses) over multiple frames, poses, and points simultaneously.
 - **Motion-only BA** optimizes only the current camera pose while keeping map points fixed, often used in tracking.
 - **Local BA** optimizes a window of recent poses and points.
- **Trajectory Incrementation (Integration):** Concatenating the estimated relative motion transformation $T_{k,k-1}$, which represents the pose change from time $k - 1$ to k , with the previously estimated trajectory. The new pose at time k , P_k , is obtained by composing transformations: $P_k = P_{k-1} \cdot T_{k,k-1}$.

This incremental estimation process effectively performs "visual dead reckoning"—estimating the current position based on the previous position and the measured motion. It provides crucial high-frequency, real-time motion estimates for immediate navigation and control tasks.

However, the reliance on incremental updates is also its primary weakness. Small inaccuracies inevitably occur in each frame-to-frame motion estimate due to sensor noise, imperfect feature tracking, violations of underlying assumptions (like scene rigidity or constant illumination), and algorithmic limitations. These small errors accumulate over time, leading to **drift**: the estimated trajectory progressively deviates from the true path. Without an external reference or a mechanism to recognize previously visited locations, this drift is unbounded and grows with the distance traveled or time elapsed.

1.3. Difference from SLAM and Visual SLAM

While often used interchangeably in casual conversation, Visual Odometry (VO) and Simultaneous Localization and Mapping (SLAM), particularly Visual SLAM (VSLAM), address related but distinct problems. Understanding their differences is crucial for appreciating their respective strengths and limitations.

- **Visual Odometry (VO)** focuses specifically on **local motion estimation**.
 - **Primary Goal:** To estimate the path or trajectory of the camera relative to its starting point.
 - **Operational Scope:** Fundamentally concerned with **local consistency**. It answers the question, "What was my motion between the last frame and the current frame?".
 - **Map Representation:** VO systems typically do not build or maintain a persistent, globally consistent map of the entire environment. They might utilize a temporary, sliding window map of recently observed features or points solely to aid robust frame-to-frame tracking, discarding older information to maintain real-time performance and bounded complexity.
 - **Core Limitation:** Subject to unbounded **drift**. Because each pose estimate builds upon the previous one, errors accumulate without correction. VO systems lack the inherent ability to recognize previously visited places and correct this accumulated error. If a robot using only VO drives in a perfect square, its estimated trajectory will likely end near, but not exactly at, the starting point, and the system has no way to know or correct this discrepancy.
- **Simultaneous Localization and Mapping (SLAM)**, on the other hand, tackles the broader problem of **global state estimation** within an unknown environment.
 - **Primary Goal:** To **simultaneously** estimate the trajectory of the agent **and** create a consistent map of the surrounding environment. Localization and mapping are treated as coupled problems.
 - **Operational Scope:** Concerned with **global consistency**. It aims to answer the question, "Where am I currently located within the larger map that I have been building?".
 - **Map Representation:** The map is a central, persistent component. It stores the estimated positions of landmarks (features, points, lines, planes, objects, etc.) and potentially the poses of key historical camera viewpoints (keyframes). This map grows as the environment is explored and is continuously refined.
 - **Defining Capability (Loop Closure):** The hallmark of a true SLAM system is its ability to perform **loop closure**. This involves recognizing when the agent has returned to a previously mapped area, even after significant travel and potential drift accumulation. Place recognition modules (often using techniques like Bag-of-Words with feature descriptors) identify candidate matches between the current view and the existing map. Geometric verification confirms these matches. A successful loop closure provides a powerful constraint: the current estimated pose and the pose associated with the matched map location must correspond to the same physical place. This constraint allows the system to:
 - Estimate the total accumulated drift around the loop.
 - Trigger a **global optimization** procedure (e.g., pose-graph optimization, which adjusts keyframe poses, or full Bundle Adjustment, which adjusts poses and landmark positions) to distribute and minimize this error across the entire trajectory and map, enforcing global consistency.

Visual SLAM (VSLAM) is simply the specialization of SLAM where the primary sensor is a camera (monocular, stereo, RGB-D, event-based, etc.).

The relationship can be summarized as: **VSLAM = VO + Loop Closure + Global Optimization**.

In common VSLAM architectures like ORB-SLAM2 and ORB-SLAM3, the VO functionality is encapsulated within the "Tracking" thread, which performs the high-frequency, frame-to-frame motion estimation relative to a local map. The "Local Mapping" thread manages and refines this local map, while the "Loop Closing" thread is responsible for detecting loops and triggering the global map correction, thereby elevating the system from simple odometry to full SLAM.

1.4. Importance in Robotics and Computer Vision

Visual Odometry, often as a component of VSLAM or VIO (Visual-Inertial Odometry), has become indispensable in numerous applications across robotics and computer vision, largely due to the ubiquity, low cost, and rich information content of cameras.

1.4.1. Importance in Robotics

For any mobile robot aiming for autonomy, knowing its position and orientation (pose) within its environment is a fundamental requirement for planning, navigation, and interaction. VO provides a crucial source of this self-localization information, particularly when external systems like GPS are unavailable or unreliable.

- **Navigation in GPS-Denied Environments:** VO is the enabling technology for robots operating indoors (factories, warehouses, homes), underground (mines, tunnels), underwater, in complex urban canyons where GPS signals are blocked or reflected, or even in extraterrestrial environments like Mars, where GPS does not exist. It provides the continuous, high-frequency motion estimates needed for tasks like path following, obstacle avoidance, and precise manipulation.
- **Cost-Effective and Versatile Sensing:** Cameras offer an unparalleled combination of low cost, small size, light weight, and low power consumption (passive sensing). This makes VO feasible for a vast range of robotic platforms, from large autonomous vehicles and industrial robots down to miniature drones (MAVs), vacuum cleaners, and wearable devices. The passive nature of cameras means they don't emit signals, avoiding interference with other sensors and making them suitable for stealth applications.
- **Foundation for Robust State Estimation (Sensor Fusion):** While pure VO can be fragile (e.g., failing in textureless corridors or during rapid motion causing blur), it forms the core of highly robust **Visual-Inertial Odometry (VIO)** systems. By tightly fusing the complementary information from a camera (rich appearance, accurate in low-dynamics) and an Inertial Measurement Unit (IMU) (high-frequency motion, robust to visual degradation, provides metric scale and gravity direction), VIO systems achieve significantly enhanced robustness and accuracy, capable of bridging periods of poor visual input. Systems like ORB-SLAM3 exemplify this tight integration, achieving high precision even during aggressive motion.

1.4.2. Importance in Computer Vision

Beyond robotics, VO provides the essential geometric understanding of camera motion required for numerous computer vision tasks aimed at interpreting, reconstructing, or augmenting the 3D world from 2D images.

- **3D Reconstruction (Structure from Motion - SfM):** Creating 3D models of objects or scenes from sequences of images fundamentally requires knowing the precise camera pose for each image. VO provides these poses in real-time or near real-time. The estimated camera trajectory acts as the geometric "scaffold" upon which the 3D structure of the scene is built through triangulation of corresponding features across multiple views. While offline SfM often achieves higher global accuracy through iterative refinement, VO provides the initial estimates and enables online reconstruction capabilities.

- **Augmented Reality (AR) and Virtual Reality (VR):** Seamlessly blending virtual content with the real world (AR) or accurately tracking a user's viewpoint within a virtual world (VR) demands extremely precise, low-latency tracking of the user's head or device pose. Modern untethered headsets rely heavily on "inside-out" tracking, typically using VIO powered by onboard cameras and IMUs. Any inaccuracy, drift, or delay in the VO tracking results in virtual objects appearing unstable, swimming, or lagging behind real-world motion, quickly breaking the sense of immersion and potentially causing nausea. The demanding accuracy requirements (millimeter-level precision) push the state-of-the-art in VO and VIO algorithms.
- **Video Enhancement and Special Effects:** Accurate VO allows for sophisticated video post-processing.
 - **Video stabilization** computationally removes unwanted camera shake by estimating the 6-DoF camera motion and then warping the video frames to counteract this motion, resulting in smoother footage.
 - In visual effects (VFX), VO (often referred to as "matchmoving" in this context) is used to track the real camera's motion on set, allowing virtual elements (CGI characters, objects, set extensions) to be realistically inserted into the scene with matching perspective and movement.

1.5. Types of Visual Odometry by Sensor

The choice of camera sensor configuration is a primary differentiator for Visual Odometry systems, significantly influencing their algorithms, performance characteristics, strengths, weaknesses, and potential applications. We can classify VO systems into three main categories based on their visual input: Monocular, Stereo, and RGB-D.

1.5.1. Monocular Visual Odometry

Principles and Challenges

Monocular Visual Odometry operates using a single camera, making it the simplest and often cheapest hardware setup. Its operation relies entirely on analyzing the apparent motion of features or pixel patterns across a sequence of 2D images captured over time.

- **Principles:** The core challenge is inferring 3D structure and motion from 2D projections. When the camera moves, objects at different depths project differently onto the image plane – this phenomenon is known as motion parallax. Monocular VO exploits this parallax.
 - **Initialization:** To begin, the system needs to observe the scene from two distinct viewpoints with sufficient baseline (translation) between them. By finding and matching features between these initial frames, the relative rotation (R) and translation direction can be estimated using epipolar geometry (solving for the Essential Matrix). The initial 3D positions of matched features can then be triangulated. Crucially, the magnitude of the translation, and thus the absolute scale of the scene, cannot be determined from these two views alone.
 - **Tracking:** Once an initial map (a set of 3D points) is established, subsequent frames are processed by tracking known features or detecting new ones. The camera's pose for the new frame is estimated by finding correspondences between the 3D map points and their 2D projections in the current image and solving the Perspective-n-Point (PnP) problem. New 3D points can be triangulated when features are reliably observed from frames with sufficient parallax.
- **Challenges:** Monocular VO inherently suffers from several major limitations:
 - **Scale Ambiguity:** This is the most fundamental challenge. Because a single camera cannot measure distance directly, the entire reconstruction (both the camera trajectory and the 3D map structure) is only determined up to an unknown scale factor. The system might correctly estimate

that it moved forward twice as far as it moved sideways, but it doesn't know if that forward motion was 1 meter or 10 meters. The scale can sometimes be resolved by incorporating additional sensors (like an IMU), recognizing objects of known size, or observing specific motion patterns (e.g., assuming constant height above a ground plane), but it is not inherent in the visual data itself.

- **Scale Drift:** Compounding the ambiguity, the estimated scale factor can drift over time. Small errors in estimating rotation and translation direction accumulate, and these errors can interact in complex ways, causing the perceived scale of the reconstruction to change as the camera moves. This results in trajectories that not only drift in position and orientation but also warp in size.
- **Initialization Difficulty:** The need for sufficient initial translation (baseline) to observe parallax makes initialization less robust than with stereo or RGB-D. If the camera starts by rotating in place or moving directly forward/backward, initialization can fail or be very inaccurate. This often necessitates specific user motion (like moving sideways) to bootstrap the system.
- **Inability to Handle Pure Rotations:** During pure camera rotations, there is no motion parallax. All points appear to move, but their relative positions in 3D cannot be determined, and new points cannot be triangulated. While the camera's orientation can still be estimated (e.g., using feature tracks and solving for a Homography), translation estimation and mapping cease.

1.5.2. Stereo Visual Odometry

Principles of Triangulation

Stereo Visual Odometry utilizes a pair of cameras, typically mounted with a fixed, known spatial relationship (the baseline, b). Often, the cameras are synchronized to capture images simultaneously, and their images are rectified so that corresponding points lie on the same horizontal scanline, simplifying the matching process.

- **Principles:** The core advantage of stereo vision is its ability to perceive depth directly through triangulation using parallax from a single instant.
 - **Stereo Matching:** For a point visible in both the left and right images, its 2D coordinates $p_L = [u_L, v_L]$, $p_R = [u_R, v_R]$ are obtained. In a rectified setup, $v_L = v_R = v$. The difference in horizontal coordinates, $d = u_L - u_R$, is called the **disparity**.
 - **Triangulation:** Given the baseline b , camera focal length f , and disparity d , the depth Z of the 3D point is computed using: $Z = \frac{f \cdot b}{d}$. Once Z is known, the full 3D coordinates (X, Y, Z) can be obtained.
- **Motion Estimation:** By performing stereo matching on each incoming pair of frames, the system obtains a 3D point cloud at each time step. The relative motion between consecutive frames can then be estimated by:
 - Tracking features in, say, the left image sequence and using their triangulated 3D positions from stereo to solve a 3D-to-2D PnP problem for the next frame.
 - Directly aligning the 3D point clouds from consecutive time steps using algorithms like ICP.
 - Performing Bundle Adjustment that minimizes reprojection errors in both left and right images across multiple views.
- Systems like ORB-SLAM2 handle stereo input efficiently. They extract features (e.g., ORB) in the left image and find their corresponding matches in the right image along the epipolar line. This provides immediate depth for many features.

Advantages and Disadvantages

- **Advantages:**
 - **Metric Scale:** Stereo VO directly recovers the metric scale of the environment and the camera trajectory from the start, thanks to the known baseline. It overcomes the scale ambiguity and significantly reduces scale drift compared to monocular VO.
 - **Robust Initialization:** Initialization is generally trivial. The first stereo frame provides enough information (a 3D point cloud) to start tracking. No special motion is required.
 - **Immediate Structure:** 3D points can be triangulated from a single stereo pair, allowing for quicker and potentially denser map building, especially for nearby objects ("close points").
- **Disadvantages:**
 - **Hardware Complexity:** Requires two cameras, precise calibration of their intrinsic parameters and relative pose (extrinsic calibration), and typically hardware synchronization.
 - **Limited Depth Range & Accuracy:** Depth accuracy from stereo triangulation is inversely proportional to the square of the distance. Faraway points have very small disparities, leading to high uncertainty in depth estimation ("far points"). The effective working range is limited by the baseline length and image resolution; longer baselines improve range but can make matching harder.
 - **Computational Cost:** Processing two image streams and performing dense or sparse stereo matching adds computational overhead compared to monocular systems.
 - **Stereo Matching Failures:** Finding correct correspondences (stereo matching) can be challenging in regions lacking sufficient texture, containing repetitive patterns, or experiencing occlusions between the left and right views.

1.5.3. RGB-D Visual Odometry

Utilizing Depth Information

RGB-D Visual Odometry leverages sensors that simultaneously capture a standard color (RGB) image and a per-pixel depth (D) map. The depth map provides a direct measurement of the distance from the sensor to the surfaces in the scene for each pixel in the depth image, usually aligned or registered to the color image.

- **Principles:** The availability of direct depth measurements dramatically simplifies the VO process:
 - **Instant 3D Point Cloud:** Each RGB-D frame directly provides a dense or semi-dense 3D point cloud of the scene by back-projecting each pixel (u, v) with depth $D(u, v)$ into 3D space using the camera intrinsics. No multi-view geometry or triangulation over time is needed for initial structure recovery.
 - **Motion Estimation Simplified:** Estimating the motion between consecutive frames becomes more straightforward:
 - **ICP:** Iterative Closest Point and its variants can be used to directly align the 3D point cloud from frame $k-1$ to the point cloud from frame k .
 - **Feature-Based with Depth:** 2D features can be detected in the RGB image, and their 3D positions can be directly looked up from the corresponding depth map. These 3D-to-2D correspondences (between 3D points from the previous frame/map and 2D features in the current frame) can be used with PnP algorithms. Alternatively, 3D-to-3D correspondences can be used.

- **Direct Methods with Depth:** Direct methods can minimize a combination of photometric error (from RGB images) and geometric error (difference between depth map values and projected map points).
- **Metric Scale:** Like stereo, RGB-D sensors provide measurements in metric units, inherently resolving scale ambiguity .

Sensor Characteristics (Kinect, RealSense)

Most consumer-grade RGB-D sensors are active, employing their own illumination source (usually infrared) to measure depth. Prominent technologies include:

- **Structured Light:** These sensors project a known pattern of infrared light (dots or lines) onto the scene. A camera captures the distorted pattern, and the deformation reveals the 3D shape and depth. Examples include the original Microsoft Kinect (v1) and Asus Xtion. They often provide dense depth but can struggle with interference from other structured light sensors or sunlight.
- **Time-of-Flight (ToF):** These sensors emit pulses of infrared light and measure the time it takes for the light to travel to the scene, reflect, and return to the sensor. This time is directly proportional to distance. Examples include Kinect v2 and some configurations of Intel RealSense LiDAR cameras. ToF can work over longer ranges than structured light but may have lower spatial resolution or be sensitive to ambient light and surface reflectivity.
- **Active Stereo:** This approach combines stereo vision principles with active illumination. An IR projector casts a pseudo-random pattern onto the scene, creating texture even on uniform surfaces. Two IR cameras capture stereo images of this pattern, and depth is computed via stereo matching algorithms. Many Intel RealSense depth cameras utilize this technology. They benefit from stereo principles but add robustness in low-texture areas thanks to the active projection.

These active sensing technologies share common characteristics influencing their use in VO:

- **Optimal Range:** They usually perform best within a specific depth range (e.g., 0.5m to 5m), with accuracy degrading outside this band.
- **Sunlight Interference:** The infrared signals can be easily washed out by strong sunlight, making most consumer RGB-D sensors primarily suitable for indoor use.
- **Surface Dependencies:** Performance suffers on highly reflective (mirrors), transparent (glass), or highly light-absorbing (dark fabric) surfaces that disrupt the IR light path.
- **Power Consumption:** Active illumination requires more power than passive cameras.

Advantages and Disadvantages

- **Advantages:**
 - **Direct Dense Depth:** Provides dense depth information without computational stereo matching, readily yielding rich 3D point clouds.
 - **Works in Textureless Regions:** Since depth is measured actively, it can often perceive the geometry of visually textureless surfaces like plain walls, where passive methods (monocular, stereo) struggle.
 - **Metric Scale:** Provides absolute scale information inherently.
 - **Algorithm Simplification:** Enables direct application of algorithms like ICP for pose estimation.
- **Disadvantages:**
 - **Limited Range:** Typically shorter effective range compared to passive visual methods.

- **Indoor Limitation:** Most sensors perform poorly outdoors due to sunlight interference.
- **Sensitivity to Materials:** Fails on certain types of surfaces (reflective, transparent, absorbing).
- **Higher Power, Size, Cost:** Generally consumes more power, is physically larger, and potentially more expensive than standard cameras.
- **Potential for Motion Artifacts:** Rapid motion might cause artifacts in depth measurements, depending on the specific sensing technology (e.g., rolling shutter effects in some sensors, or pattern smearing in structured light).

Modern SLAM systems often provide flexible interfaces to handle these different sensor types. ORB-SLAM2 and ORB-SLAM3 ingeniously treat RGB-D input by synthesizing a "virtual" right stereo coordinate from the depth value associated with each left-image feature. This allows the majority of their processing pipeline, particularly the optimization backend (Bundle Adjustment), to remain agnostic to whether the input was true stereo or RGB-D derived stereo. Similarly, deep learning approaches like DROID-SLAM can incorporate RGB-D data by adding the sensor depth measurements as constraints within their optimization framework, leveraging the additional information for improved accuracy and robustness.

1.6. Data Types and Sensor Fusion in Visual Odometry

While Visual Odometry (VO) fundamentally relies on cameras, achieving robust and accurate performance in real-world scenarios often necessitates integrating data from multiple sensor types. Different sensors possess complementary strengths and weaknesses, and fusing their measurements allows the system to overcome the limitations of any single sensor, leading to enhanced reliability, accuracy, and broader operational capabilities.

1.6.1. Key Data Types

Modern VO and state estimation systems leverage a variety of sensors, each providing distinct but often complementary information about the agent's motion and its surrounding environment.

1. Image Streams (Visual Data)

This is the foundational data type for Visual Odometry. Cameras provide rich, high-dimensional information about the appearance of the environment..

- **Monocular:** A single stream of images. Provides rich texture and structural information but suffers from inherent scale ambiguity and requires motion (parallax) to perceive depth. Fails during pure rotations and is sensitive to initialization.
- **Stereo:** Two synchronized image streams from cameras with a known baseline. Allows direct depth estimation via triangulation within a limited range, resolving scale ambiguity and simplifying initialization. Accuracy degrades with distance.
- **RGB-D:** Provides both a color image (RGB) and a per-pixel depth map (D). Offers direct, dense 3D information within its operational range, inherently resolving scale. Typically uses active sensing (e.g., structured light, time-of-flight), often limiting it to indoor environments and specific ranges/surface materials.

Visual data is excellent for recognizing features, building detailed appearance-based maps, and detecting loop closures due to its rich descriptive power. However, it is susceptible to failure in textureless environments, under challenging illumination conditions (low light, high dynamic range), and during fast motion causing motion blur.

2. Inertial Measurement Unit (IMU) Data

IMUs are electronic devices that measure and report a body's specific force, angular rate, and sometimes orientation, using a combination of accelerometers and gyroscopes. Magnetometers are sometimes included to provide a global heading reference, though they are highly susceptible to magnetic interference indoors.

- **Accelerometers:** Measure proper acceleration (including the gravity vector) in the sensor's body frame.
- **Gyroscopes:** Measure angular velocity (rate of rotation) in the sensor's body frame.

IMUs provide high-frequency (typically 100Hz to 1000Hz) motion measurements, independent of the external environment's appearance.

- **Strengths:** They work in complete darkness or textureless areas, are robust to motion blur, provide good short-term relative motion estimates, and can measure the gravity vector, which helps determine orientation (pitch/roll) and, crucially, aids in resolving the scale factor in monocular Visual-Inertial Odometry (VIO).
- **Weaknesses:** IMU measurements are noisy and contain biases that drift over time. Integrating acceleration and angular velocity to get position and orientation leads to rapid error accumulation (quadratic drift for position from acceleration). They provide no absolute position or heading reference (unless magnetometers are reliable).
- Techniques like **IMU preintegration** allow summarizing the relative motion information between visual keyframes efficiently, enabling its use in optimization-based VIO systems without needing to reprocess raw IMU data every time poses are adjusted.

3. Global Navigation Satellite System (GNSS / GPS) Data

GNSS receivers (commonly referred to by the US system, GPS) determine their position on the Earth's surface by trilaterating signals from orbiting satellites.

- **Strengths:** Provides absolute global position measurements (latitude, longitude, altitude) in an Earth-fixed frame. This is invaluable for grounding the VO/SLAM trajectory in a global coordinate system and correcting large-scale drift over long distances, especially outdoors.
- **Weaknesses:** Requires a clear view of the sky; unavailable indoors, in urban canyons, tunnels, or underwater. Measurements are typically low frequency (1-10Hz) compared to cameras or IMUs. Accuracy can be poor (meters to tens of meters for standard receivers) and is affected by atmospheric conditions, satellite geometry, and multipath reflections (signals bouncing off buildings). Techniques like RTK (Real-Time Kinematic) can improve accuracy significantly but require additional infrastructure.

4. Point Clouds

Point clouds are sets of data points in 3D space, representing the external surfaces of objects or environments. While not a direct sensor output in the same way as images or IMU readings, they are a crucial data representation derived from sensors and used within VO/SLAM.

- **Sources:** Can be obtained directly from sensors like LiDAR (Light Detection and Ranging), generated from Stereo or RGB-D camera measurements at each frame, or built up over time by triangulating features in monocular or stereo VO/SLAM systems (forming the "map").
- **Usage:** Used for motion estimation via point cloud registration algorithms (like ICP), for dense mapping, obstacle detection, and environmental modeling. LiDAR point clouds, in particular, provide accurate,

direct 3D measurements over long ranges, largely independent of ambient illumination, making LiDAR Odometry a powerful alternative or complement to VO.

1.6.2. Sensor Fusion Strategies

Given the complementary nature of these data types, fusing them within a single state estimation framework is key to building VO systems that are simultaneously accurate, robust, and capable of operating over long durations in diverse environments. The goal of sensor fusion is to combine noisy and partial information from multiple sources to obtain a better estimate of the system's state (e.g., pose, velocity, sensor biases) than could be obtained from any single sensor alone.

Motivation for Fusion

- **Robustness:** Fusing IMU data with visual data (VIO) allows the system to continue estimating motion during periods of visual degradation (e.g., fast motion, low light, textureless areas) where pure VO would fail. The IMU effectively bridges the gaps.
- **Accuracy:** Combining high-frequency relative motion from IMU with lower-frequency but drift-correcting visual updates (especially loop closures or matches to a global map) and absolute position fixes from GPS leads to trajectories that are both locally smooth and globally accurate.
- **Scale Determination:** For monocular VO, fusing with an IMU (VIO) makes the metric scale observable due to the accelerometer measuring gravity and linear accelerations in metric units. Fusing with GPS also provides metric scale.
- **Initialization:** IMU data can aid in initializing monocular VO by providing orientation (gravity direction) and potentially resolving scale faster.

Fusion Architectures

How the data from different sensors is combined defines the fusion architecture.

- **Loosely Coupled Fusion:**
 - **Concept:** Each sensor (or sensor subsystem) independently estimates the agent's state (e.g., VO estimates trajectory, GPS provides position fixes). A separate fusion filter (e.g., a Kalman filter) then combines these **state estimates**.
 - **Example:** Running an independent VO algorithm and occasionally correcting its estimated position using GPS measurements.
 - **Pros:** Simpler to implement, more modular (the VO system doesn't need to know about GPS, and vice-versa).
 - **Cons:** Suboptimal, as correlations between the errors of the different state estimates are ignored. Information is lost by processing sensor data independently first. Failure in one subsystem (e.g., VO failing) can lead to complete failure, as raw sensor data from other sources isn't used directly to bridge the gap. Error propagation can be complex.
- **Tightly Coupled Fusion:**
 - **Concept:** **Raw measurements** from **all** sensors are used directly within a **single** state estimation framework. The estimator jointly optimizes the system's state (pose, velocity, biases, map features, etc.) by minimizing the errors associated with all sensor measurements simultaneously.
 - **Example:** Visual-Inertial Odometry (VIO) systems like ORB-SLAM3, VINS-Mono, or VI-DSO, which jointly minimize visual reprojection errors and IMU preintegration errors within a single optimization problem. Fusing GPS measurements can be done by adding GPS position factors to this same optimization.

- **Pros:** Statistically optimal, as it correctly accounts for all correlations and cross-dependencies between sensor measurements. Can naturally handle failures or varying uncertainties in one sensor modality by relying more heavily on others within the joint optimization. Generally yields higher accuracy and robustness.
- **Cons:** More complex to implement, requiring careful time synchronization of sensor data and formulation of a joint state space and error function. Less modular, as the estimator needs intimate knowledge of all involved sensor models and noise characteristics.

Modern high-performance VO and SLAM systems predominantly employ tightly coupled fusion architectures due to their superior accuracy and robustness.

Estimation Techniques for Fusion

Several mathematical frameworks are used to perform the state estimation in sensor fusion:

- **Filtering Methods (Kalman Filters and variants):**
 - **Concept:** These are recursive Bayesian estimators that maintain an estimate of the system's current state (represented as a probability distribution, typically Gaussian) and update it sequentially as new sensor measurements arrive. The process involves a **prediction** step (using a motion model to propagate the state estimate forward in time) and an **update** step (using the latest sensor measurement to correct the predicted state).
 - **Examples:** Extended Kalman Filter (EKF) linearizes non-linear system dynamics and measurement models, which can introduce errors. Unscented Kalman Filter (UKF) uses a deterministic sampling approach (unscented transform) to better handle non-linearities. Multi-State Constraint Kalman Filter (MSCKF) was influential in early VIO, avoiding adding visual features to the state vector by using feature measurements over multiple frames as constraints.
 - **Pros:** Computationally efficient for real-time updates, naturally handling sequential data.
 - **Cons:** EKF linearization can lead to inconsistency. Representing the entire map in the filter state becomes computationally intractable for large-scale SLAM (though MSCKF avoids this for features). Past states are typically marginalized out, making relinearization or reprocessing of past data (e.g., for loop closure) difficult.
- **Optimization Methods (Graph-Based Optimization / Batch Estimation):**
 - **Concept:** These methods formulate the state estimation problem as a large non-linear least-squares optimization over a period of time or the entire trajectory. The system's state variables (e.g., camera poses at different times, landmark positions, sensor biases) are represented as nodes in a factor graph. Sensor measurements (e.g., visual feature reprojections, IMU preintegrated measurements, GPS positions, loop closure constraints) are represented as factors (edges) connecting these nodes, encoding the probabilistic constraints between them based on the sensor noise models. The goal is to find the set of state variable values that minimizes the sum of squared errors (Mahalanobis distances) defined by all factors in the graph. This corresponds to finding the Maximum a Posteriori (MAP) estimate of the entire state trajectory and map.
 - **Examples:** This is the dominant approach in modern high-accuracy VO/SLAM systems. Systems like ORB-SLAM3, VINS-Mono, VI-DSO, BASALT, and Kimera all rely on graph-based optimization, typically using libraries like g2o or Ceres Solver. Bundle Adjustment is a specific form of this applied to visual data.

- **Pros:** Statistically well-founded (provides MAP estimates). Handles non-linearities accurately without linearization approximations during the optimization itself (though linearization is used within iterative solvers like Gauss-Newton or Levenberg-Marquardt). Naturally incorporates measurements spanning non-sequential states (e.g., loop closures). Allows for relinearization and reprocessing of past data. Can perform both full batch optimization (offline) or incremental smoothing/mapping (online, using techniques like iSAM or sliding window optimization).
- **Cons:** Can be computationally more demanding than filtering, especially for large-scale batch optimization (though incremental methods mitigate this for real-time operation). Requires good initial guesses for iterative solvers to converge reliably.
- **Complementary Filters:**
 - **Concept:** A simpler filtering technique often used for fusing sensors with complementary frequency characteristics, particularly for orientation estimation (fusing low-frequency accelerometer/magnetometer data with high-frequency gyroscope data). It typically uses a low-pass filter for one sensor's estimate and a high-pass filter for the other, combining them to get a full-spectrum estimate.
 - **Pros:** Very simple and computationally cheap.
 - **Cons:** Less accurate and optimal than Kalman filters or optimization methods, primarily suitable for specific sub-problems like attitude estimation, not full VO/SLAM state estimation.

2. State-of-the-Art SLAM Techniques of Visual Odometry

2.1. ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras

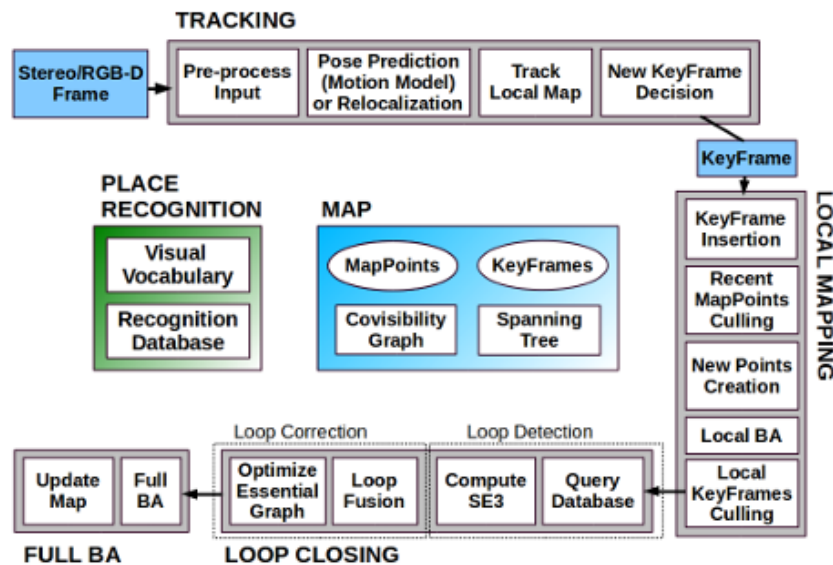
2.1.1. Introduction and System Architecture

ORB-SLAM2 is a landmark state-of-the-art Simultaneous Localization and Mapping (SLAM) system. It builds upon the successful monocular ORB-SLAM by extending its capabilities to be the first complete, open-source SLAM system to support monocular, stereo, and RGB-D cameras with a single, unified pipeline. While ORB-SLAM2 is a full SLAM system, its **"Tracking"** thread functions as the Visual Odometry (VO) component. This component is responsible for estimating the camera's pose for every new frame. The system's main contributions are its versatility, high accuracy, and real-time performance on standard CPUs.

The system's architecture is based on three parallel threads, which concurrently manage tracking, local mapping, and global optimization:

- **Tracking:** This thread performs the core visual odometry. It extracts ORB (Oriented FAST and Rotated BRIEF) features from new frames, matches them against the local map, and performs "motion-only bundle adjustment" to optimize the current camera pose. This thread is also responsible for deciding when to insert a new "keyframe" into the map.
- **Local Mapping:** This thread manages a local window of the map, co-visible with the current frame. It inserts new keyframes and map points, culls redundant ones, and performs a "Local Bundle Adjustment" (BA) to refine the map's geometry and keyframe poses in the immediate vicinity.

- **Loop Closing:** This thread is responsible for global consistency. It uses a **DBoW2** (Bag of Words) place recognition module to detect when the camera has returned to a previously mapped area (a "loop"). Upon detection, it computes a similarity transformation to correct the loop's accumulated drift, performs a pose-graph optimization, and finally launches a **Full Bundle Adjustment** (in a separate thread) to refine the entire map structure.

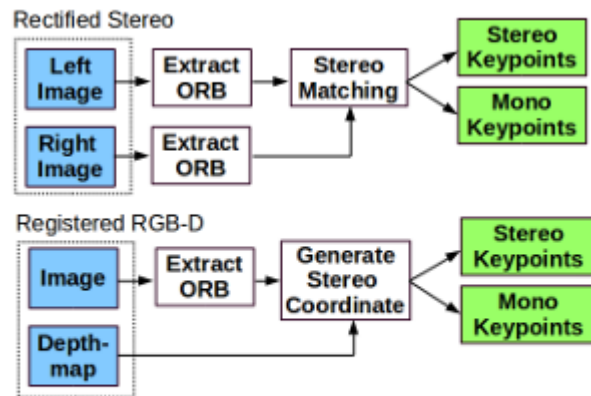


(a) System Threads and Modules.

The map itself is maintained as a **covisibility graph**, which links keyframes that observe common map points. This graph structure is fundamental, as it allows the system to scalably perform local operations (tracking and local mapping) by only considering a local neighborhood of keyframes, independent of the overall map size.

2.1.2. Key Innovation: Unification of Stereo and RGB-D Sensors

The primary innovation of ORB-SLAM2 is its elegant method for handling diverse sensor inputs (monocular, stereo, and RGB-D) within a single, unified backend. The system is designed to be agnostic to the sensor type after an initial pre-processing step. This is achieved by converting all depth-enabled inputs (stereo and RGB-D) into a common format: **"stereo keypoints."**



(b) Input pre-processing

- **For Stereo Cameras:** The system extracts ORB features from both the left and right images. For each feature in the left image, it finds the corresponding match in the right image. This is computationally efficient because stereo rectification guarantees that the match lies on the same horizontal epipolar line. The resulting stereo keypoint is defined using three coordinates: the left image coordinates (u_L, v_L) and the right image horizontal coordinate (u_R) .
- **For RGB-D Cameras:** The system first extracts ORB features from the RGB image at coordinates (u_L, v_L) . It then queries the co-registered depth map to obtain the depth value d for that feature. This depth value is used to synthesize a “virtual” right coordinate (u_R) as if it had been captured by a stereo camera with baseline b and focal length f_x : $u_R = u_L - \frac{f_x b}{d}$.

This unification is the core of the system's versatility. Once this pre-processing is complete, the tracking, mapping, and optimization threads operate on this common keypoint representation, regardless of whether the data originated from a stereo camera or an RGB-D sensor.

2.1.3. System Details: Multi-View Point Triangulation

To enhance robustness and accuracy, ORB-SLAM2's visual odometry and mapping components do not treat all points equally. The system intelligently classifies keypoints into three categories based on the reliability of their depth information:

- **Close Stereo Points:** These are key points where the depth is reliably estimated (defined as being less than 40 times the stereo baseline). These points are considered highly accurate and are triangulated immediately from a single keyframe. They are critical for providing metric scale and accurate translation estimation.
- **Far Stereo Points:** These are key points with a large depth, where the stereo disparity is small and the depth estimate is unreliable. These points are **not** triangulated immediately. Instead, they are treated like monocular points, requiring observations from multiple views to be accurately triangulated. They provide strong constraints for camera rotation but weaker information for translation.
- **Monocular Points:** These are features that only appear in the left camera image (or have no valid depth value in the RGB-D case). They are handled exactly as in the original monocular ORB-SLAM, requiring a multi-view initialization and triangulation process.

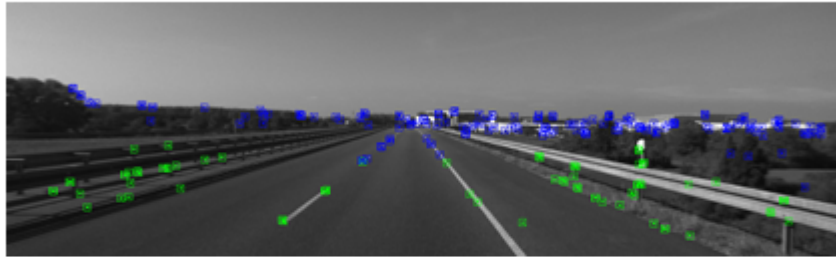


Fig. 3. Tracked points in KITTI 01 [2]. Green points have a depth less than 40 times the stereo baseline, while blue points are further away. In this kind of sequences it is important to insert keyframes often enough so that the amount of close points allows for accurate translation estimation. Far points contribute to estimate orientation but provide weak information for translation and scale.

This tri-focal point system is crucial for challenging environments like autonomous driving (as seen in the KITTI dataset), where the scene contains a mix of close objects (guard rails, road surface) and very distant points (buildings, horizon).

The system's bootstrapping is also simplified. Unlike monocular SLAM, which requires a complex initialization to estimate an initial map, the stereo/RGB-D system can bootstrap from the very first frame. It creates an initial map using all the "close stereo points" from the first keyframe, immediately establishing the metric scale of the environment.

2.1.4. Experimental Results and Conclusion

ORB-SLAM2 was evaluated on the most common and challenging public benchmarks available: KITTI (stereo), EuRoC (stereo), and TUM-RGBD (RGB-D).

- **KITTI Dataset:** The system demonstrated exceptional accuracy in large-scale driving environments. A key result, shown in the paper, is its ability to estimate the true metric scale of the trajectory without any scale drift, a significant failing of its monocular predecessor.
- **TUM-RGBD Dataset:** ORB-SLAM2 achieved state-of-the-art accuracy, outperforming contemporary systems like ElasticFusion, Kintinuous, and DVO-SLAM. The paper argues that its **Bundle Adjustment (BA)** backend, which minimizes geometric reprojection error, is fundamentally more accurate for pose estimation than methods based on Iterative Closest Point (ICP) or the minimization of photometric and depth errors. The high quality of the keyframe poses is demonstrated by back-projecting sensor depth maps to create high-definition point clouds.

2.2. DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras

2.2.1. Introduction and Core Concept

DROID-SLAM represents a radical paradigm shift in visual SLAM. Instead of relying on a traditional, hand-crafted pipeline of feature extraction, matching, and geometric optimization, DROID-SLAM introduces a

new, end-to-end deep learning architecture. The system is designed to overcome the primary weaknesses of both classic and learning-based approaches. While classic systems (like ORB-SLAM) are accurate but lack robustness, prior deep learning methods were more robust but lacked accuracy. DROID-SLAM is the first system to achieve **both state-of-the-art accuracy and state-of-the-art robustness** by integrating geometric constraints directly into a deep neural network.

The core idea is a "Differentiable Recurrent Optimization-Inspired Design" (DROID). The system maintains and iteratively refines dense, pixel-wise depth maps and camera poses through a novel **Dense Bundle Adjustment (DBA)** layer.

2.2.2. Key Innovation: Recurrent Updates and Differentiable DBA

DROID-SLAM's architecture is inspired by the RAFT (Recurrent All-Pairs Field Transforms) optical flow network. The system does not attempt to solve for pose directly. Instead, it learns to **iteratively refine** its estimates for pose and depth. The system maintains a graph of keyframes, each with a camera pose (G_t) and a dense inverse depth map (d_t).

The update process, shown in the main architecture diagram, is as follows:

1. **Correspondence Generation:** The current estimates of pose and depth are used to project a dense grid of coordinates from one frame to another, generating a dense correspondence field (optical flow).
2. **Correlation Lookup:** These correspondences are used to look up features from **4D Correlation Volumes**, which store the visual similarity between all pairs of pixels in two frames.
3. **Recurrent Update:** A **Convolutional GRU (ConvGRU)** unit receives the correlation features and current flow as input.
4. **Flow Revision:** The GRU outputs a "**flow revision**" (r_{ij}) and a **confidence weight** (w_{ij}). This is the key: the network does not predict a pose, but rather a **correction** to the current optical flow field.

This predicted flow revision is then passed to the system's most critical innovation: the **Dense Bundle Adjustment (DBA) Layer**.

This layer acts as a differentiable version of a single Gauss–Newton optimization step. Its purpose is to take the desired flow corrections predicted by the network and compute the corresponding camera pose update ($\Delta\xi$) and dense depth update (Δd). It performs a weighted non-linear least-squares optimization, where the objective is to minimize the difference between the network-predicted optical flow and the flow that would be produced by the updated pose and depth estimates.

Because this DBA layer is **fully differentiable**, the entire system can be trained end-to-end. The network learns to predict flow revisions that are not just visually plausible, but are geometrically consistent and useful for a bundle adjustment solver.

2.2.3. Training, Generalization, and System Details

The end-to-end differentiability of DROID-SLAM is its most powerful feature, as demonstrated by a critical ablation study. A system where a standard RAFT network (trained on flow) is simply attached to a **non-differentiable** BA solver ("RAFT + BA") fails completely. The DROID-SLAM ("Ours") system, trained

end-to-end **through** the DBA layer, is highly accurate. This proves that the network must learn the geometry of the problem, not just flow.

The full SLAM system operates with two threads:

- **Frontend:** Performs **local bundle adjustment** by running the update operator on a sliding window of recent keyframes.
- **Backend:** Performs **global bundle adjustment** by running the update operator over the **entire** graph of keyframes, optimizing the whole map.

The most astonishing result of the paper is the system's **generalization**. The network was trained **exclusively** on **synthetic monocular video** from the TartanAir dataset. Despite this, it can be applied at test time to:

- Real-World Monocular Video
- Real-World Stereo Video: By simply adding the right-camera frames to the graph with a fixed relative pose.
- Real-World RGB-D Video: By adding the sensor's depth as a weighted objective in the DBA layer.

This generalization is possible because the network learns the **geometry** of bundle adjustment, a universal principle, rather than overfitting to the visual appearance of a specific dataset.

2.2.4. Experimental Results and Conclusion

DROID-SLAM was evaluated against all other state-of-the-art systems on challenging benchmarks and demonstrated complete superiority in both accuracy and robustness.

- **Accuracy (EuRoC, TUM-RGBD):** DROID-SLAM significantly outperformed all competitors. On EuRoC (monocular), it reduced the error by 82% over the next best method with zero failures, and by 43% over ORB-SLAM3 (on the sequences it could complete).
- **Robustness (ETH-3D):** This benchmark is designed to test for catastrophic failures. DROID-SLAM successfully tracked **30 out of 32** sequences. The next best competitor, BAD-SLAM, tracked only 19/32, while ORB-SLAM2 tracked only 14/32. This highlights a monumental leap in robustness.

3. Industry Problems and Applications of Visual Odometry

Visual Odometry (VO) and Visual SLAM (vSLAM) are not just academic pursuits; they are mature, versatile technologies poised to address critical challenges across multiple industrial sectors, particularly in autonomous systems and immersive experiences. vSLAM, in particular, offers a camera-centric alternative to traditional sensor modalities like LiDAR and Inertial Measurement Units (IMU), providing localization and mapping capabilities that are cost-effective, passive, and information-rich.

This chapter explores the practical application areas, future trends, and specific industry implementations, including a look at how companies like Virya Autonomous Technologies are leveraging similar principles.

3.1. vSLAM Application Areas

vSLAM's utility is defined by its ability to provide robust spatial awareness using a low-cost, ubiquitous sensor: the camera. This makes it a key enabler for the following domains:

Application Area	Primary Problem Solved	vSLAM Role and Benefit
3.1.1. Autonomous Driving	Local localization for path planning and safety[cite: 593].	High-Accuracy Pose Estimation: Provides precise ego-motion and 3D environment data, which is crucial for real-time decision-making[cite: 594]. Systems like ORB-SLAM3 provide Visual-Inertial (V-I) SLAM solutions that are significantly more accurate than previous approaches[cite: 595].
3.1.2. Augmented/Virtual Reality (AR/VR)	Headset and hand tracking for immersive experiences[cite: 596].	Low-Drift Tracking: Enables precise, low-latency tracking of the user's position and orientation (6-DoF), especially indoors[cite: 597]. Modern stereo-inertial SLAM achieves errors below 1\$ cm, an excellent fit for demanding AR/VR applications[cite: 598].
3.1.3. Logistics and Warehousing	Mobile Robot (AGV/AMR) navigation and fleet management[cite: 599].	Cost-Effective Navigation: Cameras are significantly cheaper and lighter than LiDAR, providing robust navigation in complex, semi-static indoor environments like warehouses and factory floors[cite: 600].

3.2. Case Study: Virya Autonomous Technologies

Virya Autonomous Technologies serves as a relevant example of a company tackling the industrial autonomy challenge.

3.2.1. What Virya Autonomous Technologies Does

Virya Autonomous Technologies specializes in advanced autonomous solutions for industrial, logistics, and people mobility applications. Their focus is heavily weighted toward safety and reliability in their systems.

3.2.2. Primary Products and Solutions

The company's offerings are centered on Autonomous Mobile Robots (AMRs) and mobility platforms designed for seamless operation both indoors and outdoors.

- **Material Mobility Solutions:** The core business focuses on material handling equipment and robots, including:
 - **Autonomous Mobile Robots (AMRs):** Such as the AMR 10 (1-ton capacity) and AMR 50 (5-ton capacity) for moving loads within warehouses and manufacturing facilities.
 - **Autonomous Pallet Trucks.**
 - **Electric Tow Trucks** (up to 5-ton capacity).
- **Autonomous People Mobility Platforms:** Solutions for driverless transportation, including the **Autonomous Buggy** platform, which is designed for safe transportation in campuses, smart cities, and industrial parks.
- **Drive-by-Wire Research Platforms:** These are platforms used for the research and development of autonomous transportation.

3.2.3. Core Technology and Relation to vSLAM

The company emphasizes providing cutting-edge autonomous mobile robots that utilize robust navigation systems and sensor fusion technologies. The use of LiDAR for localization and navigation is consistent with the state-of-the-art navigation systems used in their Autonomous Buggy and AMRs. This LiDAR-centric approach is a common, robust solution in the industry.

The parent group, Virya Electric Powertrains (founded in 2017), focuses on electric powertrains and customized vehicle integration. This expertise provides a solid foundation for the autonomous vehicle platforms offered by Virya Autonomous Technologies. While their current public-facing material emphasizes LiDAR, the problems they solve—localization, navigation, and safety—are the same ones vSLAM aims to address, often as a complementary or redundant system to LiDAR.

3.3. Localization and Path Planning in Industry

3.3.1. Overview of the Core Problem

The localization required for autonomous mobility, whether for an autonomous car or a warehouse AMR, involves computing a vehicle's 3D pose (position and orientation) with centimeter-level accuracy within a map. This information is the foundational input for path planning and safety systems, as it allows the vehicle to:

- Understand its surroundings and establish a sense of road and lane structures.
- Make decisions on actions like changing lanes, making turns, or stopping.
- Plan future maneuvers by referencing information beyond its current Field of View (FoV) using a high-definition (HD) map.
- Identify and localize surrounding stationary and moving objects (pedestrians, other vehicles, etc.) to maneuver safely without human interaction.

3.3.2. Common Industrial Approach (e.g., Virya)

While the specific, detailed localization strategy for any given company (like Virya) is often proprietary and not fully disclosed, the general industry context points to a two-step approach:

1. **Precise Localization (The Input):** The system must first precisely locate the vehicle and its environment using sensors. To achieve the necessary accuracy and redundancy for safe operation, autonomous vehicles typically use a **fusion of multiple sensors**, including LiDAR, cameras (vision), and inertial sensors (IMU). The use of LiDAR and inertial/satellite navigation (GNSS/IMU) is standard practice, especially for localizing the vehicle within an HD map. This aligns with the sensor modalities used in the buggy data collection platform.

2. **Autonomous Path Planning (The Output):** The planning system uses this localization data to generate safe and optimal motion commands. Path planning is a key technology that uses the localized environmental model to find the most optimal, collision-free route from a current state to a target state. Safe planning is achieved by continuously evaluating the environment, predicting the behavior of dynamic objects, and generating feasible driving modes to avoid collisions and complete maneuvers smoothly. Companies like Virya specifically commit to solutions that ensure 100% safe operations.

3.4. Future Trends and Challenges in Industrial Applications

Integrating vSLAM into robust industrial platforms, either as a primary localizer or as a redundant system, requires aligning with future trends while proactively addressing known challenges.

3.4.1. Future Trends

- **Integration of Semantic SLAM:** This involves integrating semantic segmentation (i.e., identifying classes like "person," "car," "road") into the SLAM pipeline. This allows the system to distinguish dynamic objects (e.g., pedestrians, moving cars) from the static environment. This greatly improves localization robustness by masking out unreliable features from moving objects, leading to safer interaction and better contextual awareness.
- **Ultra-low power Edge Computing:** The deployment of VO/vSLAM on mobile and wearable devices is a significant driver. The efficiency of systems like SVO (which achieves high FPS on embedded platforms) and efforts to simplify deep learning architectures (e.g., lightweight convolutional networks) directly support deployment on devices with tight power and computational constraints.
- **Fusion with 5G/Cloud Infrastructure:** This trend moves beyond a single agent. Multi-map SLAM (e.g., Atlas) and Map Merging capabilities allow multiple agents (like a fleet of autonomous buggies or AMRs) to collaborate. They can share map data and build large-scale, globally consistent maps that persist across sessions, improving the efficiency and robustness of the entire fleet.

4. Paper Survey: Sensors, Algorithms, and Dataset

4.1. Sensors for Visual Odometry

The choice of sensor is a fundamental aspect of any Visual Odometry (VO) system, profoundly influencing its capabilities, limitations, algorithmic design, and application domains. While various sensors can contribute to robotic perception, VO specifically leverages cameras to interpret motion through visual changes. This section delves into the primary camera types used in VO, detailing their operating principles, interfacing methods, calibration requirements, and synchronization considerations.

4.1.1. Monocular Cameras

A monocular camera setup, employing just a single camera, represents the simplest, most cost-effective, and often most compact sensor configuration for VO. It mimics a single human eye, capturing a 2D projection of the 3D world. Despite its simplicity, extracting metric motion information presents unique challenges.

Working Principle: Pinhole Model and Image Formation

The fundamental model used to describe image formation in most monocular cameras is the pinhole camera model. This idealized model provides a mathematical framework for understanding how light from the 3D world projects onto the 2D image sensor.

- **The Pinhole Concept:** Imagine a light-proof box with a tiny hole (the pinhole or aperture) on one side and a light-sensitive surface (the image plane or sensor) on the opposite side. Light rays from a point in the 3D scene travel in straight lines. Only rays passing through the pinhole reach the image sensor. Each point in the 3D world maps to a unique point on the image plane, forming an inverted image.
- **Mathematical Representation:** To work with this model mathematically, we introduce a coordinate system.
 - **Camera Coordinate System:** A 3D coordinate system (X_c, Y_c, Z_c) is fixed to the camera, with its origin at the camera's optical center. The Z_c axis points forward along the optical axis, X_c points horizontally, and Y_c points vertically.
 - **Image Coordinate System:** A 2D coordinate system (u, v) is defined on the image plane, typically with the origin at the top-left corner and units in pixels.
- **Projection:** Consider a 3D point P with coordinates (X_c, Y_c, Z_c) in the camera frame. According to the pinhole camera model, this point projects onto the image plane at coordinates (x, y) relative to the principal point. From similar triangles:

$$x = f \frac{X_c}{Z_c}$$

$$y = f \frac{Y_c}{Z_c}$$

- Here, f is the focal length, representing the distance between the optical center and the image plane. This projection collapses 3D geometry onto a 2D plane, causing the depth Z_c to be lost and creating the inherent scale ambiguity in monocular vision
- **From Ideal to Pixel Coordinates:** The ideal image coordinates (x, y) are converted to pixel coordinates (u, v) in digital images by incorporating the principal point (c_x, c_y) and using focal lengths f_x, f_y expressed in pixel units:

$$u = f_x \frac{X_c}{Z_c} + c_x$$

$$v = f_y \frac{Y_c}{Z_c} + c_y$$

The parameters f_x, f_y, c_x, c_y collectively form the **intrinsic camera parameters**.

- **Lens Distortion:** Real camera lenses deviate from the ideal pinhole model, introducing distortions that warp the image. The most common types are:
 - **Radial Distortion:** Causes straight lines in the world to appear curved in the image, especially near the edges. It can be barrel distortion (bulging outwards) or pincushion distortion (curving inwards).
 - **Tangential Distortion:** Occurs if the lens is not perfectly parallel to the image sensor. Correcting for these distortions by estimating distortion coefficients is crucial for accurate VO.

Monocular Visual Odometry (VO) algorithms track the movement of these projected points (u, v) over time and use geometric constraints such as epipolar geometry or PnP to estimate the camera's 3D motion (R, t) and the underlying 3D structure (X_c, Y_c, Z_c) , though only up to an unknown scale factor.

Software Interface

Accessing image data involves interacting with drivers and utilizing computer vision libraries.

- **Operating System Drivers:** On Linux, the prevalent standard is **Video4Linux2 (V4L2)**. V4L2 provides a common API for applications to discover, query, configure, and capture frames from video devices.
- **Computer Vision Libraries (OpenCV):** High-level libraries like **OpenCV (Open Source Computer Vision Library)** abstract away the driver complexities. OpenCV's **VideoCapture** class provides a unified interface to access cameras across different operating systems, handling frame grabbing and parameter setting. Most VO systems rely heavily on libraries like OpenCV.
- **Proprietary SDKs:** Camera manufacturers sometimes provide their own Software Development Kits (SDKs) offering more direct control over hardware features, but may tie the application to specific hardware.

Calibration

Accurate VO requires a precise mathematical model of the camera, obtained through calibration. This process estimates the intrinsic parameters and distortion coefficients.

- **Intrinsic Parameters:**
 - **Focal Lengths (f_x, f_y) :** Represent the focal length scaled by the pixel size in the horizontal and vertical directions.
 - **Principal Point (c_x, c_y) :** The pixel location where the camera's optical axis intersects the image plane.
- **Distortion Coefficients:**
 - **Radial Distortion $(k_1, k_2, k_3, \dots)$:** Coefficients for the polynomial model correcting barrel or pincushion distortion.
 - **Tangential Distortion (p_1, p_2) :** Coefficients correcting for lens misalignment.
- **Calibration Procedure:** Calibration is typically performed offline using a known calibration target, such as a **checkerboard pattern**. The process involves capturing multiple images of the target from various positions and orientations. Software (e.g., OpenCV's calibration module) detects the known pattern points (e.g., checkerboard corners) in each image and solves an optimization problem to find the intrinsic and distortion parameters that best explain the observed 2D locations.

Synchronization

Timing is critical in VO, especially when fusing with other sensors like an IMU.

- **Software Timestamps:** The simplest approach is to record the system clock time when the image frame is received by the application. This method is easy but suffers from variable delays in communication, OS scheduling, and buffering. This "timing jitter" can be tens of milliseconds and degrades VIO accuracy.
- **Hardware Timestamps/Triggers:** More precise timing requires hardware-level synchronization.
 - **Hardware Timestamps:** Some cameras provide timestamps generated closer to the actual image exposure time.

- **Hardware Triggers:** This is the most robust method for multi-sensor systems. A common master clock (e.g., microcontroller) sends precisely timed electrical pulses to the camera(s) and IMU. Cameras in trigger mode start their exposure immediately upon receiving the pulse. This ensures that images and IMU readings are captured at precisely known moments, minimizing timing uncertainty down to the microsecond level.

4.1.2. Stereo Cameras

Stereo camera systems overcome the fundamental scale ambiguity of monocular VO by using two cameras simultaneously capturing images from slightly different, known viewpoints. This mimics human binocular vision.

Working Principle: Epipolar Geometry, Disparity, and Depth Estimation

The core principle is triangulation. A 3D point projects to different locations in the left and right images due to the separation (baseline). Knowing the camera parameters and the 2D locations in both images allows for 3D reconstruction.

- **Epipolar Geometry:** This describes the geometric relationship between the two views.
 - **Baseline:** The vector connecting the optical centers of the two cameras.
 - **Epipolar Lines:** For a point in one image, its corresponding point in the other image must lie on a specific line (the epipolar line). This reduces the search space for matches from 2D to 1D.
 - **Essential Matrix (E) / Fundamental Matrix (F):** Algebraic representations 3×3 capturing the epipolar geometry. The Essential Matrix is for calibrated cameras and encodes the relative rotation and translation.
- **Stereo Rectification:** This is a preprocessing step that computationally warps both images so that the epipolar lines become collinear and horizontal. This is done by making the image planes coplanar and the optical axes parallel. This simplifies the correspondence problem: for a point (u_L, v_L) in the left image, its match must lie on the same row $v_R = v_L$ in the right image.
- **Disparity:** In a rectified stereo pair, the disparity d is the difference in horizontal coordinates:

$$d = u_L - u_R$$
 Disparity is **inversely proportional** to the depth Z of the corresponding 3D point.
- **Stereo Matching:** This is the process of finding corresponding pixels to compute the disparity map. It is a challenging problem due to ambiguity in textureless or repetitive regions, occlusions, and non-Lambertian surfaces.
 - **Local Methods:** Fast, but prone to errors. Use a small window (patch) for matching (e.g., Block Matching, SAD)
 - **Global Methods:** More accurate and smooth, but computationally expensive. Formulated as an energy minimization problem (e.g., Graph Cuts).
 - **Semi-Global Methods:** A trade-off between accuracy and speed. **Semi-Global Block Matching (SGBM)** is a popular example.
- **Depth Calculation:** Once disparity d is known, the depth Z of the 3D point can be computed using:

$$Z = \frac{f \cdot b}{d}$$
 where f is the focal length and b is the baseline.

This equation highlights the **inverse relationship** between depth and disparity.

- **Accuracy and Limitations:** Depth resolution (ability to detect changes in depth) decreases quadratically with distance. This means stereo is accurate for nearby objects but becomes very inaccurate for far

objects. VO systems like ORB-SLAM2/3 handle this by classifying points as "close" (reliable depth) or "far" (treated as monocular).

Software Interface

- **OpenCV Stereo Functions:** OpenCV provides a comprehensive suite for stereo processing:
 - `cv::stereoCalibrate`: For estimating intrinsic and extrinsic parameters.
 - `cv::stereoRectify`, `cv::initUndistortRectifyMap`, `cv::remap`: For performing rectification.
 - `cv::StereoBM`, `cv::StereoSGBM`: Implementations of stereo matching algorithms.
 - `cv::reprojectImageTo3D`: Converts a disparity map directly into a 3D point cloud.
- **Proprietary SDKs:** Manufacturers of integrated stereo cameras (e.g., Stereolabs ZED, Intel RealSense Stereo) provide dedicated SDKs. These often offer optimized, GPU-accelerated depth map generation, factory calibration, and built-in VO/VSLAM functionalities.

Calibration

Stereo calibration is more involved, as it requires finding the parameters of both cameras and their spatial relationship.

- **Parameters to Estimate:**
 - **Intrinsics (Left Camera):** $f_{xL}, f_{yL}, c_{xL}, c_{yL}$ and distortion coefficients.
 - **Intrinsics (Right Camera):** $f_{xR}, f_{yR}, c_{xR}, c_{yR}$ and distortion coefficients.
 - **Extrinsics (Relative Pose):** Rotation matrix R and translation vector T defining the pose of the right camera relative to the left.

4.1.3. RGB-D Cameras

RGB-D cameras provide two co-registered data streams: a standard 2D color (RGB) image and a 2D depth (D) map. The depth map provides a direct distance measurement for each pixel, immediately solving:

- **Scale Ambiguity:** The depth map provides metric measurements from the first frame.
- **Initialization:** An instant 3D point cloud is available, allowing VO to start immediately without requiring parallax.

Most RGB-D cameras are **active** sensors, meaning they emit their own light (typically infrared).

Working Principle: Active Depth Sensing

Unlike passive stereo which finds correspondences, active sensors create them or use light measurement.

1. Structured Light (SL):

- **Concept:** Projects a known, pre-defined pattern of infrared light onto the scene. A calibrated IR camera, offset from the projector, captures an image of how this pattern deforms over objects.
- **Mechanism:** The system compares the *observed* pattern with the *known* projected pattern and uses triangulation to calculate depth for each point. This is like stereo vision, but one camera is replaced with a projector.
- **Examples:** Microsoft Kinect v1, Orbbec Astra.

- **Characteristics:** Can produce dense, accurate depth at short ranges (0.5-4m). However, the IR pattern is easily "washed out" by sunlight, making it unsuitable for outdoor use. Struggles with IR-absorbing (dark) or reflective (shiny) surfaces.

2. Time-of-Flight (ToF):

- **Concept:** Measures depth by timing how long light takes to travel from the sensor to the object and back. With the speed of light c , distance is computed as: $Z = \frac{c \cdot \Delta t}{2}$.
- **Mechanism (Types):**
 - **Indirect ToF (iToF):** Emits a modulated wave of light and measures the *phase shift* of the returning wave, which is proportional to distance.
 - **Direct ToF (dToF):** Emits a short pulse of light and measures the *time interval* Δt directly with a high-speed timer for each pixel. This is how LiDAR sensors work.
- **Examples:** Microsoft Kinect v2 (iToF), Intel RealSense L515 (dToF).
- **Characteristics:** Generally less susceptible to ambient light than SL. Still struggles with transparent or specular surfaces. iToF can be confused by "multipath" interference (light bouncing off multiple surfaces).

3. Active Stereo:

- **Concept:** An enhancement of the passive stereo principle, designed to solve its main weakness: textureless regions.
- **Mechanism:** A standard, calibrated pair of IR cameras observe the scene. An IR pattern projector (often pseudo-random dots) illuminates the scene. The *only* purpose of the projector is to "paint" artificial texture onto uniform surfaces. The system's software then performs standard stereo matching on the IR images.
- **Examples:** Intel RealSense D400 series (e.g., D415, D435).
- **Characteristics:** Very robust. Can handle textureless surfaces well (indoors). Can still function outdoors in sunlight; the IR projector fails, but the system simply reverts to being a *passive* stereo camera using natural light. This makes it one of the most versatile sensor types. Accuracy is still governed by stereo principles (degrades quadratically with distance).

Software Interface

Interfacing with RGB-D cameras almost exclusively relies on vendor-provided SDKs.

● Key SDKs:

- **Intel RealSense SDK (librealsense):** A comprehensive, cross-platform, open-source SDK for all modern RealSense cameras.
- **Microsoft Kinect SDK:** Separate SDKs for different Kinect generations. The Azure Kinect SDK provides access to the ToF sensor, RGB camera, and a built-in IMU.
- **OpenNI (Open Natural Interaction):** An older, open-source framework, pivotal for the original Kinect but less common for modern devices.

● Core SDK Functionality for VO:

- **Stream Management:** Initiating and capturing synchronized data streams, often as a **frameset** (a container with aligned RGB, depth, and IMU frames).
- **Frame Alignment:** This is a critical function. Since the RGB and depth sensors are physically separate, the SDK provides functions (e.g., `rs2::align`) to **align** the depth frame to the color frame's viewpoint. This uses the pre-calibrated extrinsics to re-project the depth map, resulting in

a depth image where every pixel (u,v) directly corresponds to the color pixel (u,v) . This is the data most VO systems consume.

- **Point Cloud Generation:** Many SDKs provide on-the-fly functions to convert a depth map into a 3D point cloud.

VO systems like ORB-SLAM2/3 link against these SDKs, request aligned frames, and then process them. For example, ORB-SLAM2 extracts a feature at (u, v) in the RGB image and simply looks up the depth at the *same* (u, v) coordinate in the aligned depth map to compute its virtual disparity.

Calibration

RGB-D calibration is a complex, multi-stage process, though it is usually performed at the factory and stored on the device's firmware.

- **Individual Sensor Intrinsic:** Both the RGB camera and the depth sensor (e.g., the IR camera) must be calibrated to find their own intrinsic parameters and distortion models.
- **Extrinsic Calibration (Depth-to-RGB Alignment):** This is the most critical part. The system must know the precise 6-DoF Rotation: R Translation: T between the RGB sensor and the depth sensor. This is effectively a stereo calibration problem, solved by observing a checkerboard in both the RGB and IR views simultaneously to compute the fixed transformation between them. The SDK uses R and T to perform the frame alignment.

Synchronization

Synchronization for an RGB-D camera is almost entirely an internal hardware and firmware-level concern, which is a significant advantage.

- **Internal Hardware Sync:** The components (projector, IR sensor, RGB sensor, and often IMU) are integrated onto a single circuit board, driven by a common clock.
- **SDK-Managed Framesets:** The SDK abstracts this away, providing the application with a complete, synchronized "frameset" where all frames (color, depth, IMU) share a common hardware timestamp. This built-in, reliable synchronization is a major reason for their popularity in VO/SLAM.

4.1.4. Lidar (Brief Mention in Context of Fusion)

Lidar (Light Detection and Ranging) is another active sensing technology central to robotics, and Lidar Odometry (LO) is a field in its own right. Lidar is also increasingly fused with cameras in VO systems.

- **Working Principle:** Like dToF cameras, Lidar works on the principle of direct Time-of-Flight. It emits a laser pulse and measures the time Δt for it to return, calculating distance $Z = \frac{c \cdot \Delta t}{2}$.
- **Point Cloud Generation:** Lidar builds its point cloud by **scanning**.
 - **2D Lidar:** A single laser/sensor spins in a 2D plane.
 - **3D Lidar:** A vertical stack of multiple lasers (e.g., 16, 32, 64) is spun 360 degrees, generating a full 3D point cloud at 10-20 Hz.
- **Key Differences from RGB-D:**
 - **Range:** Lidar has a much longer range (e.g., 50-200m) compared to most RGB-D cameras (e.g., 0.5-10m).

- **Robustness:** Lidar is almost completely unaffected by ambient illumination, working equally well in total darkness and bright sunlight.
- **Output:** The native output is a 3D point cloud, which is often sparser than a dense RGB-D map but covers a much larger volume.
- **Software Interface:** Relies on vendor drivers/SDKs and the **Point Cloud Library (PCL)**, which is the de-facto standard for 3D point cloud processing (analogous to OpenCV for 2D images). **ROS (Robot Operating System)** is also heavily used, with standard message types for point clouds.
- **Calibration (for Lidar-Visual Fusion):** For fusion, the system *must* know the rigid-body transformation (extrinsic calibration (R, T)) between the Lidar and the camera. This complex calibration is often done using specialized targets (like large checkerboards) visible to both sensors and tools like Kalibr.
- **Synchronization (for Lidar-Visual Fusion):**
 - **Internal Motion Distortion (Skewing):** A 3D Lidar scan is not instantaneous; it takes time (e.g., 100ms) to spin 360 degrees. If the robot moves during this time, the point cloud will be **distorted** or "skewed".
 - **Motion Compensation:** To correct this, Lidar Odometry systems *must* use a high-frequency motion estimate (almost always from an IMU) to perform **motion compensation**, transforming each Lidar point back to the pose at the *start* of the scan. This requires extremely precise time synchronization between the Lidar and the IMU.
 - **External Synchronization (PPS):** High-end systems (Lidar, IMU, GPS) often accept a **Pulse Per Second (PPS)** signal from a GPS receiver. This provides a highly accurate, shared time reference for all sensors, which is the gold standard for Lidar-Inertial fusion.

4.2. Sensor Calibration and Synchronization

For a Visual Odometry system to function, it must operate on a foundation of known, reliable data. Raw sensor measurements are imperfect; cameras distort light, and inertial sensors drift. When using multiple sensors, the system must also know precisely *where* they are relative to each other and *when* they captured their data. Sensor calibration and synchronization build this foundation. They are arguably the most critical prerequisites for any high-performance VO, VIO, or multi-sensor fusion system.

- **Calibration** is the process of building an accurate mathematical model of a sensor. It corrects imperfections and translates raw measurements into a geometrically meaningful format. It answers, "How does this sensor see the world?".
- **Synchronization** is the process of aligning all data streams to a single, common timeline. It answers, "When did the sensors see this?".

Errors in either do not merely reduce accuracy; they introduce fundamental inconsistencies that can cause estimation algorithms to diverge, leading to catastrophic tracking failure.

4.2.1. Methods for Camera Calibration (Intrinsic)

Camera calibration is the process of determining the camera's internal geometric and optical characteristics, its **intrinsic parameters**. The goal is to find the parameters for the pinhole camera model that accurately link 3D geometry to 2D image measurements.

Parameters to be Estimated:

- **Focal Length** (f_x, f_y): Represent the lens's focal length in pixel units, scaling 3D coordinates to 2D image-plane coordinates.
- **Principal Point** (c_x, c_y): The 2D pixel coordinate where the camera's optical axis intersects the image plane, typically near the image center.
- **Distortion Coefficients**: Model the non-ideal behavior of real lenses.
 - **Radial Distortion** ($k_1, k_2, k_3, \dots$): Models the most significant distortion (barrel or pincushion), which causes straight lines to appear curved.
 - **Tangential Distortion** (p_1, p_2): Models smaller distortions from lens misalignment.

Calibration Procedure: Using Known Patterns The standard method involves observing a calibration target with a precisely known 3D geometry from multiple viewpoints..

- **Calibration Target**: The algorithm must know the exact 3D coordinates of feature points on a target. Common targets include:
 - **Chessboard**: The classic target. The corners of the squares can be detected with very high sub-pixel accuracy. The 3D coordinates of all corners are known (e.g., (0,0,0), (20,0,0) in mm).
 - **ArUco Markers**: Fiducial markers with unique binary codes. They are individually identifiable, so the algorithm knows the 3D-to-2D correspondence unambiguously, even if the pattern is partially occluded.
 - **ChArUco (Chessboard + ArUco)**: A hybrid pattern combining the best of both. ArUco markers are placed in the white squares, providing robust identification, while the high-precision chessboard corners are used for the final calibration.
- **Image Capture**: The user captures 15-20 images of the pattern from a wide variety of viewpoints, distances, and orientations, ensuring the pattern is visible near the image edges to properly observe lens distortion.
- **Feature Detection**: For each image, an algorithm (e.g., `findChessboardCorners`) finds the 2D pixel coordinates of the pattern features.
- **Optimization**: The algorithm runs a non-linear optimization (e.g., Levenberg-Marquardt) to find the single set of intrinsic parameters and distortion coefficients that minimizes the total **reprojection error**. The reprojection error is the pixel distance between the *detected* 2D points and the *predicted* 2D points (calculated by projecting the known 3D points using the candidate parameters). The result is a **camera matrix** and a **distortion vector**.

4.2.2. IMU Calibration (Intrinsic)

A low-cost MEMS IMU (3-axis accelerometer, 3-axis gyroscope) is not a perfect sensor. Its raw measurements are corrupted by deterministic and stochastic errors. For VIO, the sensor *must* be calibrated to model these errors.

1. Deterministic Errors (Biases and Scale Factors):

- **Bias (Offset)**: A constant offset $\vec{b}_g$ in the sensor's output. Even when still, the gyro reports a non-zero velocity $\vec{\omega}_{measured} = \vec{\omega}_{true} + \vec{b}_g$.
- **Scale Factor Error**: The output is not perfectly scaled (e.g., a 90° turn measures as 90.1°).
- **Axis Misalignment**: The physical X, Y, Z axes are not perfectly orthogonal.

- **Calibration (Accelerometer):** A simple method is a **6-point static test**. The IMU is held still in 6 orientations (each axis pointing up, then down). In each state, it should only measure Earth's gravity (1g). By comparing the 6 measurements to the known gravity vector, one can solve for the bias and scale/misalignment matrix.

2. Stochastic Errors (Noise):

These are random, time-varying errors characterized statistically. They are the primary reason IMU integration drifts.

- **Measurement Noise (White Noise):** High-frequency, zero-mean Gaussian noise on every measurement, characterized by the **Noise Density** parameter.
- **Bias Instability (Bias Random Walk):** The "constant" bias is not constant; it drifts slowly and unpredictably over time, modeled as a random walk. This is the most problematic error for VIO. It is characterized by the **Bias Random Walk** parameter.

Calibration Procedure (Allan Variance)

The noise parameters are found using the Allan Variance method.

- **Procedure:** The IMU sensor must be left perfectly stationary for several hours, collecting data at high frequency.
 - **Analysis:** The collected data is analyzed to see how its variance changes with different averaging time intervals. A log-log plot of this Allan variance vs. averaging time produces a characteristic "U-shaped" curve.
 - **Interpretation:** The noise parameters can be read directly from this plot. The value at a slope of -1/2 corresponds to the Noise Density. The minimum point corresponds to the Bias Instability, and the value at a slope of +1/2 corresponds to the Bias Random Walk.
- These noise parameters are essential inputs for the VIO algorithm (e.g., Kalman filter or optimizer), which uses them to model its trust in the IMU's integrated measurements over time.

4.2.3. Multi-Sensor Extrinsic Calibration

Extrinsic calibration finds the fixed 6-DoF rigid-body transformation (rotation R and translation T) *between* the coordinate frames of two different sensors. This is a fundamental requirement for all multi-sensor fusion.

1. Camera-to-Camera (Stereo Calibration):

- As described in section 4.1.2, this is a form of extrinsic calibration. The `stereoCalibrate` function solves for the intrinsics of both cameras and the single R and T defining the pose of the right camera relative to the left.

2. Camera-to-IMU Calibration:

- This is one of the most critical and difficult calibrations for VIO, finding $T_{C \leftarrow I'}$
- **Method (e.g., Kalibr toolbox):** This process relies on motion. The core idea is to observe a static calibration target (e.g., checkerboard) with the camera-IMU rig *while the rig is moving*.
- **Data Collection:** The rig is moved to excite all 6 DoF (especially rotations). The system records:
 1. **Camera Data:** The pose of the camera relative to the target $T_{C \leftarrow Target}$ for each timestamp.
 2. **IMU Data:** Raw gyro/accel readings between the same timestamps.

- **Optimization:** A large graph-based optimization finds the $T_{C \leftarrow I}$ that makes the motion measured by the

$$T_{C \leftarrow C_{k-1}} \approx T_{C \leftarrow I} T_{I \leftarrow I_{k-1}} T_{I \leftarrow C}$$

3. Lidar-to-Camera Calibration:

- This is essential for fusing Lidar point clouds with camera images, finding $T_{C \leftarrow L}$
- **Method:** Uses a calibration target, typically a large checkerboard, captured from several positions.
- **Data Collection:**
 1. **Camera Data:** The 2D pixel coordinates of the checkerboard's corners.
 2. **Lidar Data:** The 3D points from the Lidar scan that lie on the flat plane of the checkerboard.
- **Optimization:** This is a 3D-to-2D problem. The algorithm knows the 3D corner coordinates (from Lidar) and the 2D corner coordinates (from the image) and solves for the $T_{C \leftarrow L}$ that minimizes the reprojection error of these 3D-to-2D correspondences.

4.2.4. Time Synchronization Techniques

Time is the most critical component of sensor fusion. An error of 1ms between an IMU and a camera rotating at 100 deg/sec creates a 0.1° error, which is non-recoverable and can cause the estimator to fail. The goal is to stamp all sensor measurements with a timestamp from a single, common, high-precision clock.

1. Software Timestamps (Arrival Time):

- **Method:** This is the simplest and worst approach. The host computer timestamps data when it *arrives* at the software (e.g., after USB transport).
- **Problem:** This timestamp records arrival time, not capture time. It is corrupted by large, variable, and unknown latency (sensor buffering, data transmission, OS driver buffering, OS thread scheduling). This "timing jitter" makes precise fusion impossible.

2. Software-based Network Time Synchronization (NTP, PTP):

- **NTP (Network Time Protocol):** Can synchronize host computer clocks over a network to millisecond-level.
- **PTP (Precision Time Protocol):** A much more precise version, achieving sub-millisecond or microsecond-level synchronization of host clocks over Ethernet.
- **Limitation:** These only synchronize the *computers*, not the *sensors*. They do not solve the variable transport and OS latency problem.

3. Hardware Synchronization (The Correct Solution):

- **Method:** This approach bypasses software timestamps and uses electrical signals to coordinate all sensors.
- **Mechanism (Hardware Trigger):**
 1. A single, central **master clock** (microcontroller, FPGA, or GPS receiver) generates a periodic electrical pulse (e.g., a 30 Hz square wave)
 2. This single wire is split and connected to the "external trigger" or "sync" input pins on the camera(s), IMU, and Lidar.
 3. When the rising edge of this pulse occurs, all sensors act *at that exact instant* (within nanoseconds). The camera begins its exposure, and the IMU latches its current reading.

4. The data packets then travel to the computer with their usual variable latencies, but the data *itself* is now "stamped" internally with the *exact time* or *trigger number* of the event.
- Result: The software can now perfectly align all measurements. It can match image #100 with IMU reading #100, knowing they happened at the same time, regardless of when the packets arrived.
 - **PPS (Pulse Per Second):** A common example of a master clock signal, often provided by a GPS receiver. It delivers a highly accurate pulse at the beginning of each UTC second. This is the gold standard for synchronizing outdoor systems, as it provides both a common trigger and a link to global time.

For any high-performance VO system involving an IMU (VIO) or other sensors, hardware synchronization is not an optional refinement; it is a fundamental requirement for stable and accurate operation.

5. Autonomous Buggy vehicle

5.1. Autonomous Buggy Platform Overview

The autonomous buggy platform (code name: GITAM Autonomous Vehicle (GAV)) is a specialized, multi-sensor vehicle designed for high-precision autonomous mobility and, crucially, for the collection of high-fidelity sensor data. The system architecture emphasizes redundancy and high-throughput data processing to ensure safety and reliability across industrial, campus, or controlled urban environments.

While the platform's foundation for current navigation relies on a LiDAR-Centric SLAM (Simultaneous Localization and Mapping) approach, its role in this research is as a sophisticated **data collection tool**. The high-resolution VLP-32C LiDAR generates a persistent, metric map used for centimeter-level localization, which provides high-quality ground-truth odometry. This ground truth is recorded in parallel with data from the visual-inertial sensors, creating rich, synchronized **ROS bag files** for the development and testing of Visual Odometry (VO) and Visual SLAM (VSLAM) algorithms

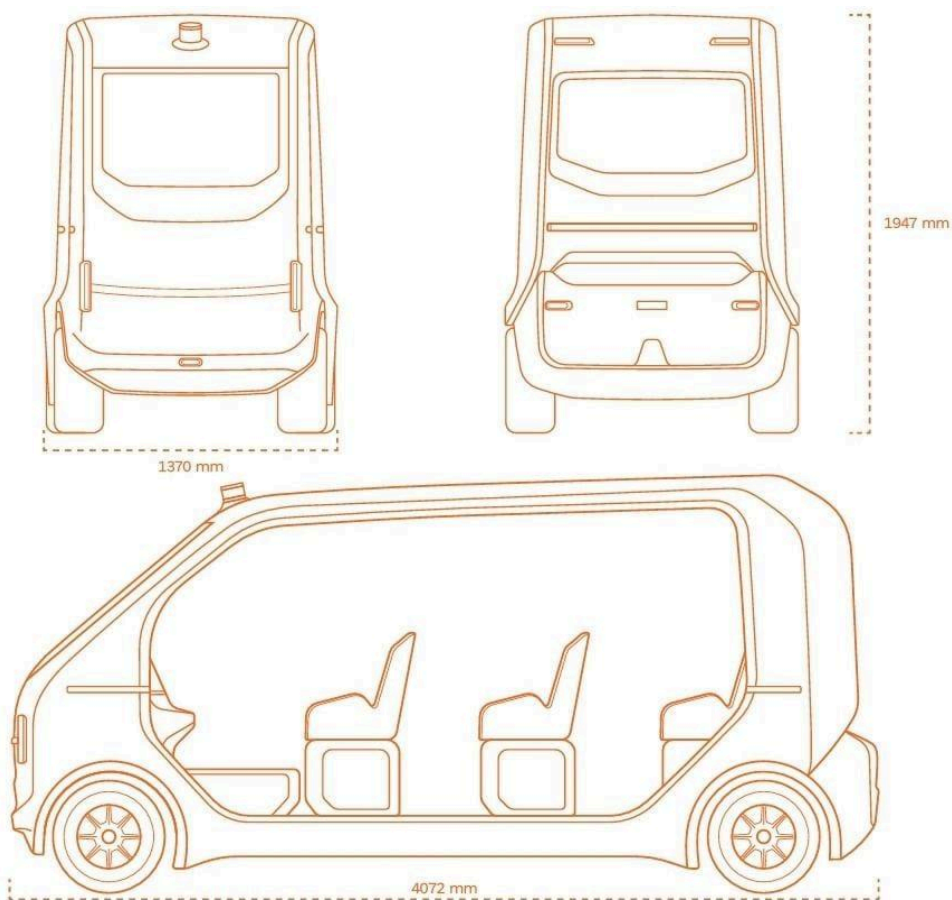
5.2. Vehicle Dimensions and Mechanical Specifications

The vehicle is built on a heavy-duty, electric platform with precise specifications essential for accurate kinematic modeling and control.

	Parameter	Specification	Purpose in Autonomy
Vehicle Class	People/Material Buggy (Approx 5 Ton Class)	High stability required for passenger/material transport.	High stability required for passenger/material transport.
Overall Length (L)	4072 mm	Used for global path planning and motion control constraints.	Used for global path planning and motion control constraints.
Overall Height (H)	1947 mm	Determines sensor placement height and vertical clearance checks.	Determines sensor placement height and vertical clearance checks.

Overall Width (W)	1370 mm	Critical for lane keeping , tight maneuver planning, and 2D LiDAR safety zones.	Critical for lane keeping , tight maneuver planning, and 2D LiDAR safety zones.
--------------------------	---------	--	--

Vehicle Dimensional Drawing:



5.3. Sensor and Perception Stack: Detailed Configuration

The system integrates a diverse array of sensors, fusing 3D mapping with high-frequency inertial and safety inputs, all of which are recorded into the ROS data bus.

5.3.1. Primary Localization and Visual-Inertial (V-I) Sensors

These sensors provide the continuous, high-accuracy data required for SLAM and pose estimation, forming the core of the data collection effort.

Sensor	Model	Key Specifications	Localization Role
--------	-------	--------------------	-------------------

3D Mapping LiDAR	Velodyne VLP-32C	32 Channels, 200 m Range, 360 deg FoV	Metric SLAM Baseline: Generates the persistent 3D map and provides primary localization via Point Cloud Registration.
Visual SLAM Camera	Intel RealSense D435i	90 fps Depth Rate, Wide FoV, Onboard IMU	VSLAM Development Input: Primarily used to collect high-fidelity ROS bags (stereo, RGB-D, IMU, and real odometry) for offline development of Visual SLAM (VSLAM), Visual Odometry (VO), and Object Detection algorithms.
High-Precision IMU	Xsens MTI-610	6-DoF, Accelerometers, Gyroscopes, Magnetometers	Drift Suppression: Fuses with visual/LiDAR data to provide high-frequency attitude and velocity estimates, improving odometry accuracy.

5.3.2. Safety, Object Detection, and Kinematics Sensors

These sensors provide auxiliary input for local awareness and odometry verification, which is also logged in the ROS bags.

Sensor	Model	Key Specifications	Placement & Function
2D Safety LiDAR	SICK TIM581-2050101	25 m Range, 270 deg Scan Angle	Obstacle Avoidance: Three units (2 front, 1 rear) dedicated to monitoring the

			immediate vehicle perimeter for collision risk.
Wheel Encoder	Autonics Absolute Encoder (EP50S-6)	Absolute Digital Output, 12-24VDC	Odometry Baseline: Provides non-slip distance measurement, vital for odometry calculation and velocity estimation.

5.4. Sensor Placement and Visualization

Accurate calibration of the extrinsic parameters (relative position/orientation between sensors) is fundamental for sensor fusion. The sensors are placed to maximize coverage and minimize self-occlusion.

5.4.1. External Sensor Placement Overview

Sensor	Placement Position	Purpose of Placement
VLP-32C	Top-Center on Roof	Maximize 360 deg long-range FoV, minimizing occlusion by the vehicle body. Crucial for large-scale mapping.
2D SICK LiDARs	Low on Front/Rear Bumpers	Maximize low-level coverage for detecting ground-level obstacles, curbs, and people's legs immediately near the vehicle.
RealSense Camera	Front Chassis/Bumper Level	Provides a complementary view to the 2D LiDARs for visual perception and depth measurement in the direction of travel.

Vehicle and Sensor Placement Diagram:



Buggy Configuration



SICK TiM581-2050101



MTI-610 IMU



Velodyne VLP32C

5.4.2. Detailed Sensor Images (Descriptions Only)

Sensor/Component	Description
Propulsion Motor	GITAM Autonomous Vehicle (GAV) A.C. Motor: Showing 4.0 kW, 48V specifications.
Motor Controller	CURTIS A.C. Motor Controller: Showing 48V, 375A specifications.
Inertial Sensor	Xsens MTI-610 IMU: Used for high-rate attitude tracking.
Wheel Encoder	Autonics Absolute Encoder: Attached near the axle for motion measurement.

5.5. Actuation, Control, and Computational Environment

5.5.1. Actuation and Power Subsystems

Subsystem	Component	Key Specifications	Control Link
Propulsion	GITAM Autonomous vehicle (GAV) A.C. Motor (4.0 kW)	Voltage: 48 V; Frequency: 138 Hz	Controlled by CURTIS Controller, receiving speed commands from ROS.
Steering	MTU BLDC Motor (1.2 kW)	Peak Power: 1000 W; Voltage: 72 V	Receives precise angle commands from the Motion Planning module.
Power	SHIZEN Energy Li-ion Pack	48.1 V, 100 Ah Capacity; 200 A Discharge	Provides stable, high-current power required for the motors and the demanding computing platform.

5.5.2. Current Navigation Strategy: The LiDAR-SLAM Workflow

The buggy's current localization method, which serves as the ground truth for data collection, relies exclusively on **LiDAR data** and **A-priori mapping**.

1. **A-Priori Mapping (Offline):** The VLP-32C is used to record the environment and build a dense, persistent 3D map. This map includes known static features that serve as anchors.
2. **Waypoint Generation:** Key paths, goals, and stops are recorded as waypoints linked to the metric map.
3. **Real-Time Localization (SLAM):** The vehicle uses incoming VLP-32C point clouds to perform LiDAR SLAM (specifically, localization against the map). This process continuously computes the vehicle's 6-DoF pose by matching live point cloud data against the stored map.
4. **Object Detection and Safety:** The 2D SICK LiDARs provide the crucial safety function: detecting immediate static and dynamic obstacles.
5. **Path Execution:** The localization pose is fed to the planning software, which generates steering and velocity commands to follow the planned waypoints.

5.5.3. Visual Data Collection for Development

While the vehicle currently *relies* on LiDAR-SLAM for real-time operation, the **Intel RealSense D435i** is continuously used to **collect synchronized ROS bags**.

This data collection serves as the foundation for the development and integration of Visual SLAM (VSLAM), Visual Odometry (VO), and advanced visual Object Detection algorithms. The ROS bags contain a complete, multi-modal dataset:

- Stereo image streams (from D435i)
- RGB-D data (from D435i)
- IMU data (from D435i and Xsens MTI-610)
- High-precision ground-truth odometry (from the LiDAR-SLAM and Wheel Encoder)

This setup provides an ideal testbed for developing and validating new visual localization algorithms against a highly accurate baseline.

5.5.4. Software Environment

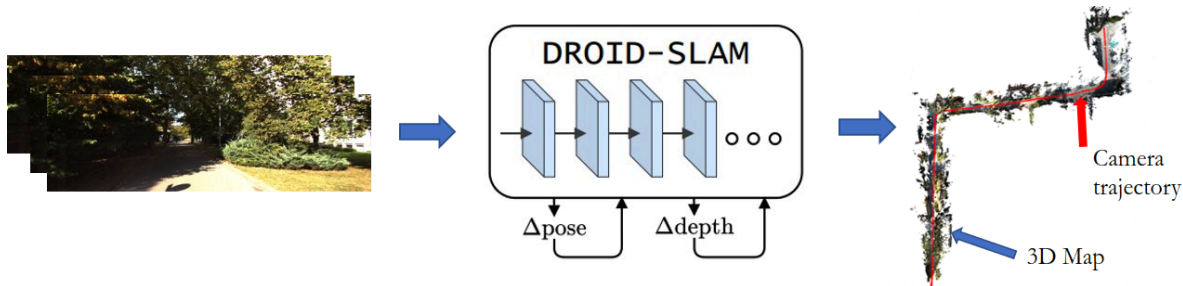
The high complexity of the sensor fusion and control loops mandates a powerful industrial computing unit (ThinkEdge SE50) supported by specialized software libraries.

Software Component	Purpose in Pipeline	Localization Implication
ROS	Middleware / Data Bus	Synchronizes all sensor inputs (LiDAR, IMU, RealSense, Encoder) and manages the execution of localization nodes.
CUDA	Real-Time Acceleration	Essential for running computationally expensive algorithms like point cloud processing (ICP) and will be critical for accelerating Visual-Inertial SLAM (ORB-SLAM3) integration.
MATLAB	Offline Simulation / Tuning	Used for refining control laws, kinematic models, and tuning noise parameters for the Xsens IMU.
Localization Algs	LiDAR SLAM Algorithms	The primary software responsible for localization and map management.

6. Implementations and Simulations on Visual Odometry

6.1 DROID-SLAM Overview

DROID-SLAM (Differentiable Recurrent Optimization-Inspired Design for SLAM) is an end-to-end neural architecture combining:

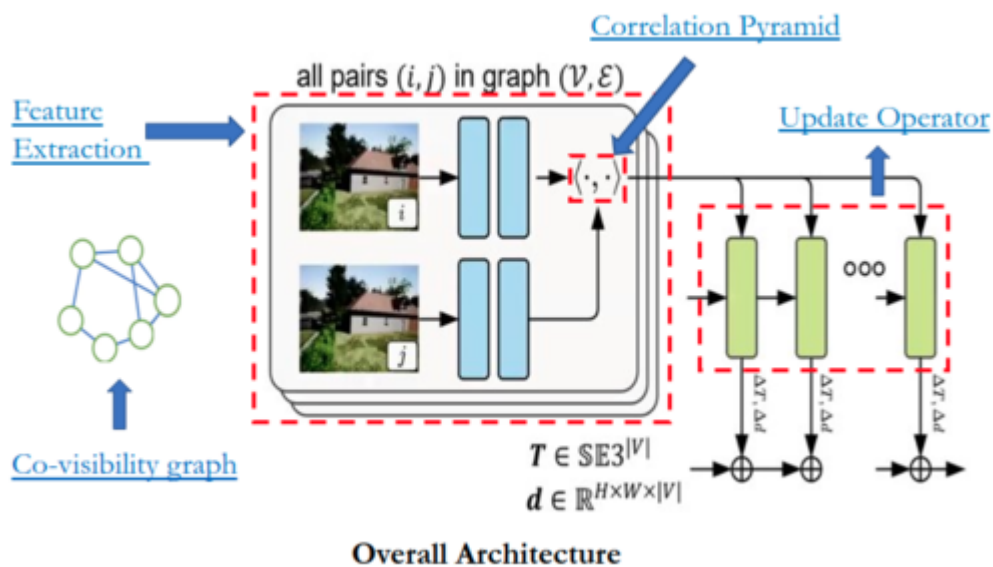


DROID SLAM OVERVIEW

- Recurrent iterative updates (inspired by RAFT)
- Differentiable Dense Bundle Adjustment (DBA)

It estimates camera poses and pixel-wise depths through recurrent updates, enabling accurate 3D reconstruction even from monocular video. At inference, it can also leverage stereo or RGB-D inputs for enhanced accuracy.

6.1.1 Architecture



DROID-SLAM operates on an ordered set of images. Each frame maintains two state variables:

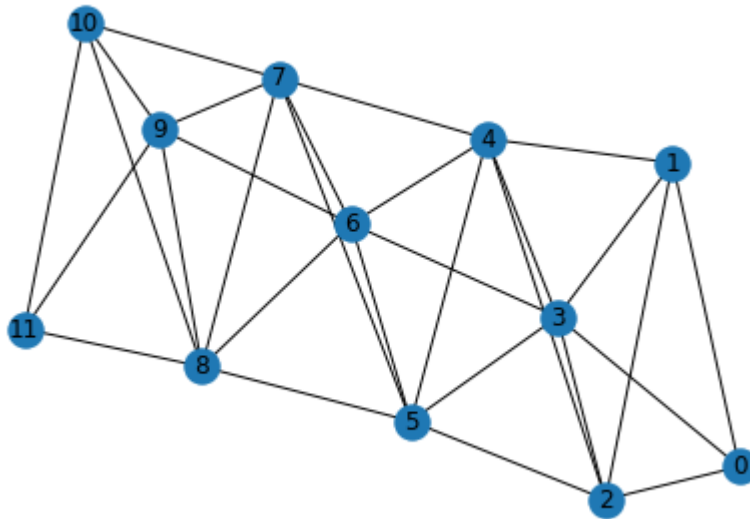
- Camera pose
- Inverse depth

These states are iteratively refined as new frames arrive.

6.1.2 Co-Visibility Graph

Frames are represented as nodes in a **frame graph**, where edges indicate co-visible regions.

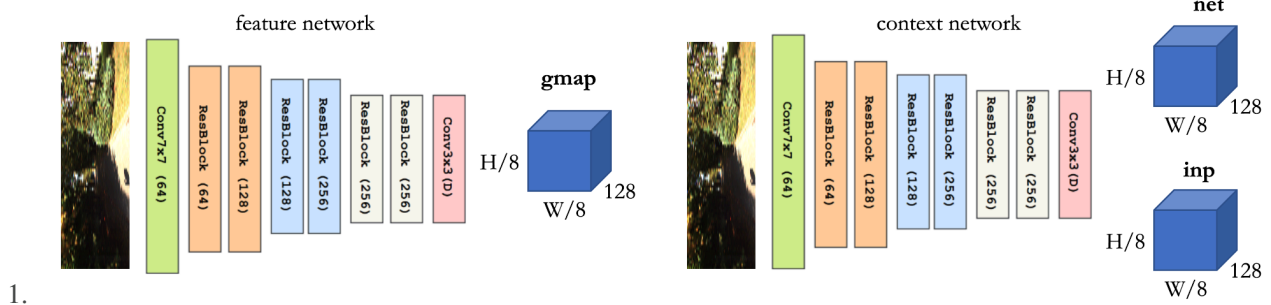
- Graphs are updated dynamically during training and inference.
- Loop closure is achieved by connecting revisited regions to earlier nodes.



6.1.3 Feature Extraction

Two networks process each frame:

- **Feature Encoder** (128 channels, with instance normalization)
- **Context Encoder** (256 channels, no normalization)

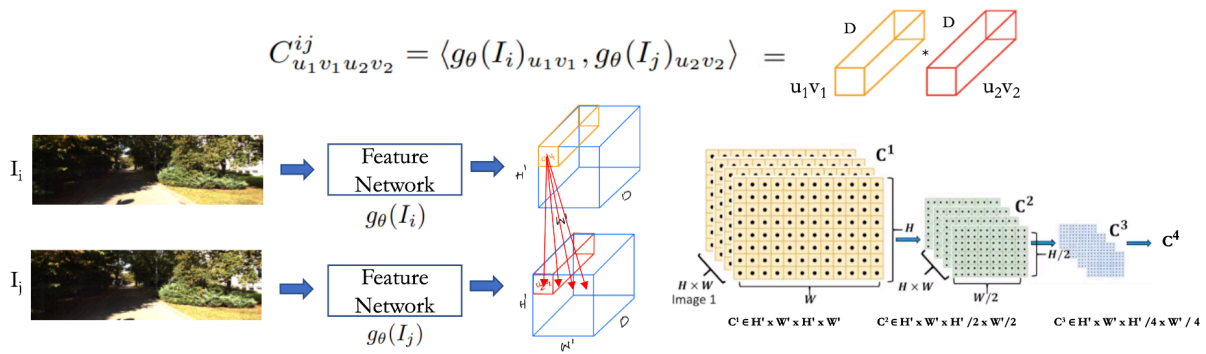


Feature and Context Networks of DROID-SLAM

Each consists of six residual blocks and three downsampling layers. Feature maps build correlation volumes, while context features guide the update operator.

6.1.4 Correlation Pyramid

For each pair of co-visible frames, a 4D correlation volume is computed as the dot product of all feature pairs. Subsequent average pooling across scales yields a 4-level correlation pyramid, used during pose and depth refinement.



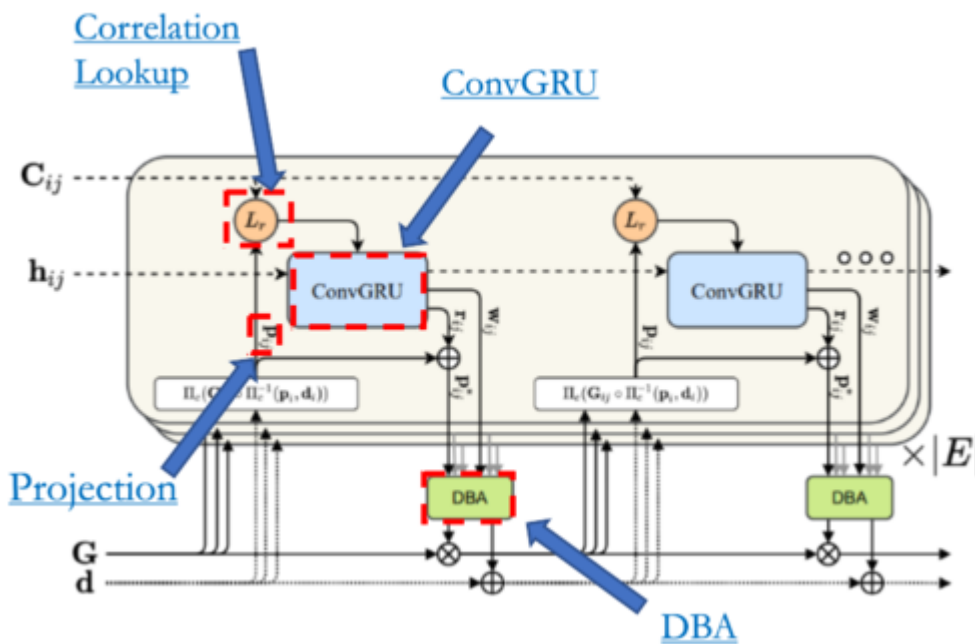
Correlation Pyramid

6.1.5 Update Operator

A 3x3 convolutional GRU acts as the core update unit. Each iteration:

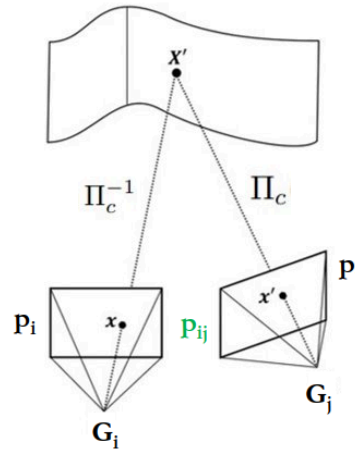
- Updates the hidden state
- Outputs pose and depth corrections
- Applies them via manifold retraction (for pose) and vector addition (for depth)

Repeated updates converge toward accurate estimates of camera trajectory and scene depth.



6.1.6 Correspondence Field

Given current depth and pose estimates, DROID-SLAM computes the dense correspondence field—mapping pixel coordinates from one frame to another using estimated transformations. This flow, induced by camera motion, represents the geometric relationship between frames.



$$\mathbf{p}_{ij} = \Pi_c(\mathbf{G}_{ij} \circ \Pi_c^{-1}(\mathbf{p}_i, \mathbf{d}_i)), \quad \mathbf{p}_{ij} \in \mathbb{R}^{H \times W \times 2} \quad \mathbf{G}_{ij} = \mathbf{G}_j \circ \mathbf{G}_i^{-1}$$

$$\Pi_c(\mathbf{X}) = \begin{pmatrix} f_x \frac{X}{Z} + c_x \\ f_y \frac{Y}{Z} + c_y \\ 1 \\ d \end{pmatrix} \quad \Pi_c^{-1}(\mathbf{p}, d) = \begin{pmatrix} \frac{p_x - c_x}{f_x} \\ \frac{p_y - c_y}{f_y} \\ 1 \\ d \end{pmatrix}$$

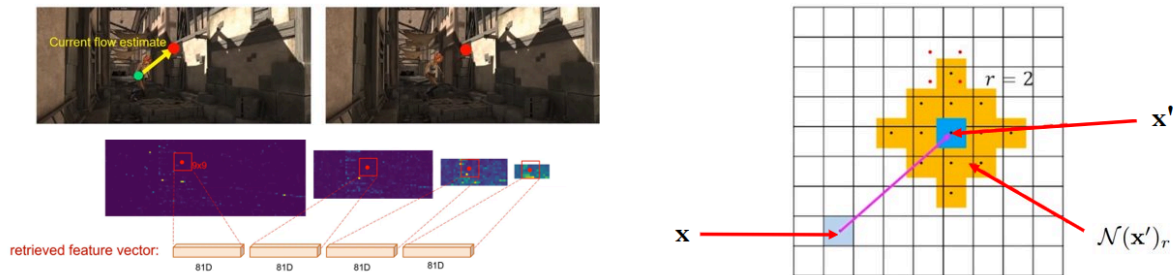
Dense Correspondence field

6.1.7 Correlation Lookup

The lookup operator retrieves features from the correlation pyramid based on estimated correspondences. A local grid around each correspondence point is sampled across pyramid levels to enhance robustness against motion scale and depth uncertainty.

$$L_r : \mathbb{R}^{H \times W \times H \times W} \times \mathbb{R}^{H \times W \times 2} \mapsto \mathbb{R}^{H \times W \times (r+1)^2}$$

$$\mathbf{x} = (u, v) \quad \mathbf{x} \in I_1 \quad \mathbf{x}' = (u + f^1(u), v + f^2(v)) \quad \mathbf{x}' \in I_2 \quad \mathcal{N}(\mathbf{x}')_r = \{\mathbf{x}' + \mathbf{dx} \mid \mathbf{dx} \in \mathbb{Z}^2, \|\mathbf{dx}\|_1 \leq r\}$$



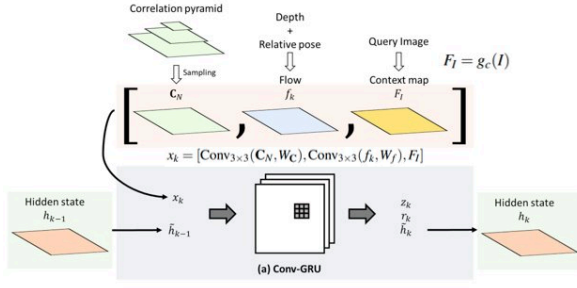
Lookup Operator

6.1.8 ConvGRU Layer

The GRU refines the hidden state and predicts:

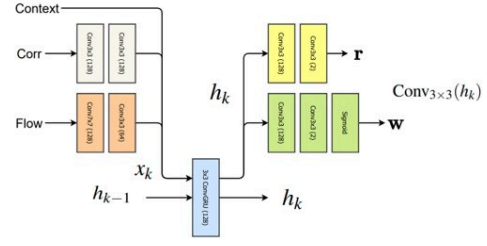
- A revision flow field
- A confidence map

These outputs guide the Dense Bundle Adjustment process by correcting correspondence errors.



$$\mathbf{p}_{ij}^* = \mathbf{r}_{ij} + \mathbf{p}_{ij}$$

$\mathbf{p}_{ij}^*$ corrected dense correspondence field



$$\begin{aligned} z_k &= \sigma(\text{Conv}_{3 \times 3}([h_{k-1}, x_k], W_z)) \\ r_k &= \sigma(\text{Conv}_{3 \times 3}([h_{k-1}, x_k], W_r)) \\ \tilde{h}_k &= \tanh(\text{Conv}_{3 \times 3}([r_k \odot h_{k-1}, x_k], W_h)) \\ h_k &= (1 - z_k) \odot h_{k-1} + z_k \odot \tilde{h}_k \end{aligned}$$

6.1.9 Dense Bundle Adjustment (DBA) Layer

The DBA layer converts flow corrections into pose and depth updates. It minimizes the Mahalanobis-weighted reprojection error using Gauss-Newton optimization with pixelwise damping. Efficient computation is achieved via the Schur complement. This differentiable optimization layer enables joint learning of geometry and motion.

cost function over the entire frame graph

$$\mathbf{E}(\mathbf{G}', \mathbf{d}') = \sum_{(i,j) \in \mathcal{E}} \|\mathbf{p}_{ij}^* - \Pi_c(\mathbf{G}'_{ij} \circ \Pi_c^{-1}(\mathbf{p}_i, \mathbf{d}'_i))\|_{\Sigma_{ij}}^2 \quad \Sigma_{ij} = \text{diag } \mathbf{w}_{ij}$$

$$\begin{bmatrix} \mathbf{B} & \mathbf{E} \\ \mathbf{E}^T & \mathbf{C} \end{bmatrix} \begin{bmatrix} \Delta \xi \\ \Delta \mathbf{d} \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \mathbf{w} \end{bmatrix} \quad \begin{aligned} \Delta \xi &= [\mathbf{B} - \mathbf{E}\mathbf{C}^{-1}\mathbf{E}^T]^{-1}(\mathbf{v} - \mathbf{E}\mathbf{C}^{-1}\mathbf{w}) \\ \Delta \mathbf{d} &= \mathbf{C}^{-1}(\mathbf{w} - \mathbf{E}^T \Delta \xi) \end{aligned}$$

$$\begin{aligned} \mathbf{G}^{(k+1)} &= \text{Exp}(\Delta \xi^{(k)}) \circ \mathbf{G}^{(k)} & \text{camera pose } \mathbf{G}t \in \text{SE}(3) & \{\mathbf{G}^{(k)}\} \rightarrow \mathbf{G}^* \\ \mathbf{d}^{(k+1)} &= \Delta \mathbf{d}^{(k)} + \mathbf{d}^{(k)} & \text{inverse depth } \mathbf{d}t \in \mathbb{R}_+^{H \times W} & \{\mathbf{d}^{(k)}\} \rightarrow \mathbf{d}^* \end{aligned}$$

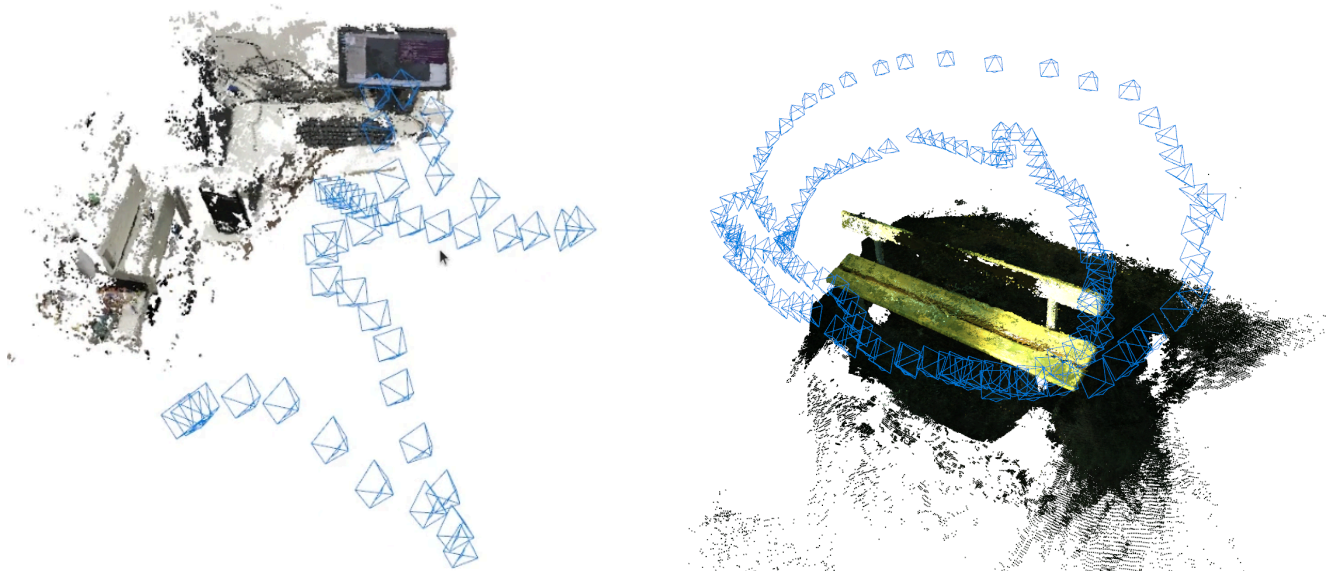
DBA Layer

6.1.10 Inference

The SLAM system processes a video stream as input and performs real-time reconstruction and localization. It operates using two asynchronous threads to manage computational load efficiently. The **frontend thread** is responsible for acquiring new frames, extracting features, selecting keyframes, and performing local bundle adjustment (BA). Simultaneously, the **backend thread** performs global bundle adjustment over the entire set of keyframes to refine overall trajectory consistency.

Although DROID-SLAM can, in principle, execute both frontend and backend operations concurrently, the Lenovo SE70 with NVIDIA Jetson Xavier has limited GPU resources. Therefore, in this setup, the system runs these components asynchronously—with tracking and local BA executed on the GPU first, followed by global BA and loop closure once the frontend completes.

This configuration successfully handles around 1600 images in a single run, requiring a GPU with at least 11 GB of memory for stable real-time performance.



Monocular 3D Reconstruction

simulation on KITTI Dataset:

 Droid_kitti_seq_01_map.mp4

 DROID_slam_gitam_map.mp4

6.2 Lunar Rover Simulation and Navigation Stack

6.2.1 Overview 6.2.2 System Architecture Flow

This project details the development and implementation of an autonomous navigation system for a simulated lunar rover. The architecture is built using the Robot Operating System (ROS 2) and the Gazebo simulator.

The system first performs 2D mapping of an unknown environment using slam_toolbox with 2D LiDAR data. Once a static map is generated and saved, the rover utilizes the Nav2 stack for autonomous localization (AMCL), path planning, and navigation, including the handling of dynamic obstacles.

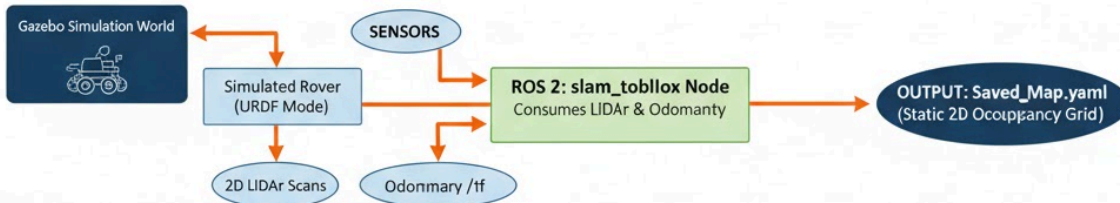
The primary limitation of this 2D-plane system is its inability to navigate the uneven and complex 3D terrain characteristic of a lunar environment. The future work focuses on integrating VSLAM and 3D LiDAR to create a robust 3D mapping and navigation solution.

6.2.2 System Architecture Flow

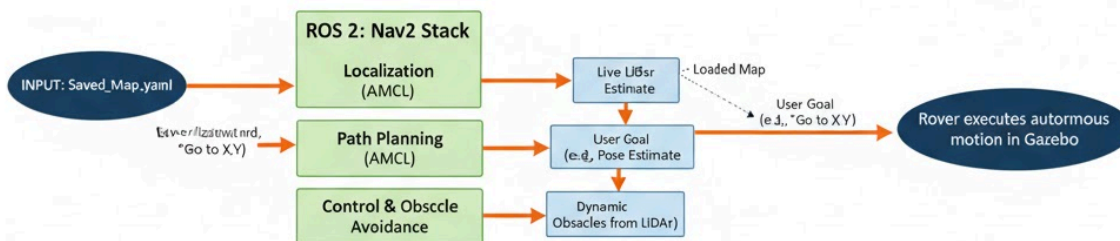
Here is a high-level data and process flow for the system's phases:

6.2.2.2 System Architecture Flow

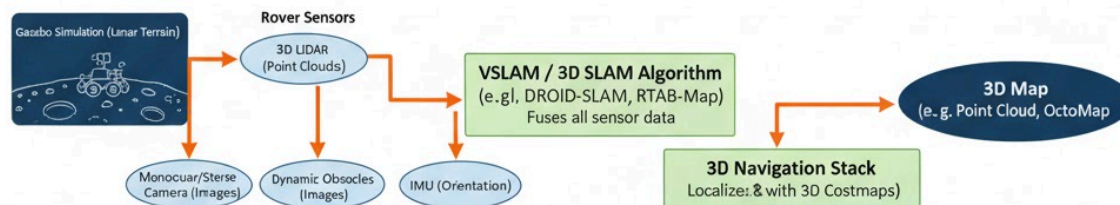
Phase 1: 2D Mapping (Data Collection & SLAM)



Phase 2: 2D Navigation (Localization & Path Planning)



Future Work: 3D VSLAM & Navigation



6.2.3 Simulation Environment and Robot Model

The rover and its environment are modeled and simulated in Gazebo, a 3D robotics simulator that provides realistic physics and sensor feedback.

- **Robot Model (URDF):** The rover's physical structure, including its chassis, wheels, and sensor attachments, is defined in a URDF (Unified Robot Description Format) file. All sensors (cameras, LiDAR, IMU) are attached to the robot's base link via fixed or movable "joints," allowing for accurate pose data.
- **World Model:** A custom Gazebo "world" file defines the simulation environment, including static obstacles and terrain features.

6.2.4 Sensor Configuration

The simulated rover is equipped with a sensor suite designed for 2D and 3D perception:

- **2D LiDAR:** Provides planar laser scans (sensor_msgs/LaserScan) used by slam_toolbox and Nav2 for 2D mapping and obstacle avoidance.
- **IMU (Inertial Measurement Unit):** Provides orientation and angular velocity data to improve odometry estimates.
- **Wheel Encoders:** Publish joint states which are used by robot_state_publisher and odometry nodes.
- **Monocular/Stereo Cameras:** Included in the model to provide image streams (sensor_msgs/Image)

intended for the future VSLAM implementation.

6.2.5 2D SLAM and Mapping

Mapping of the planar simulation environment is achieved using the ROS 2 slam_toolbox.

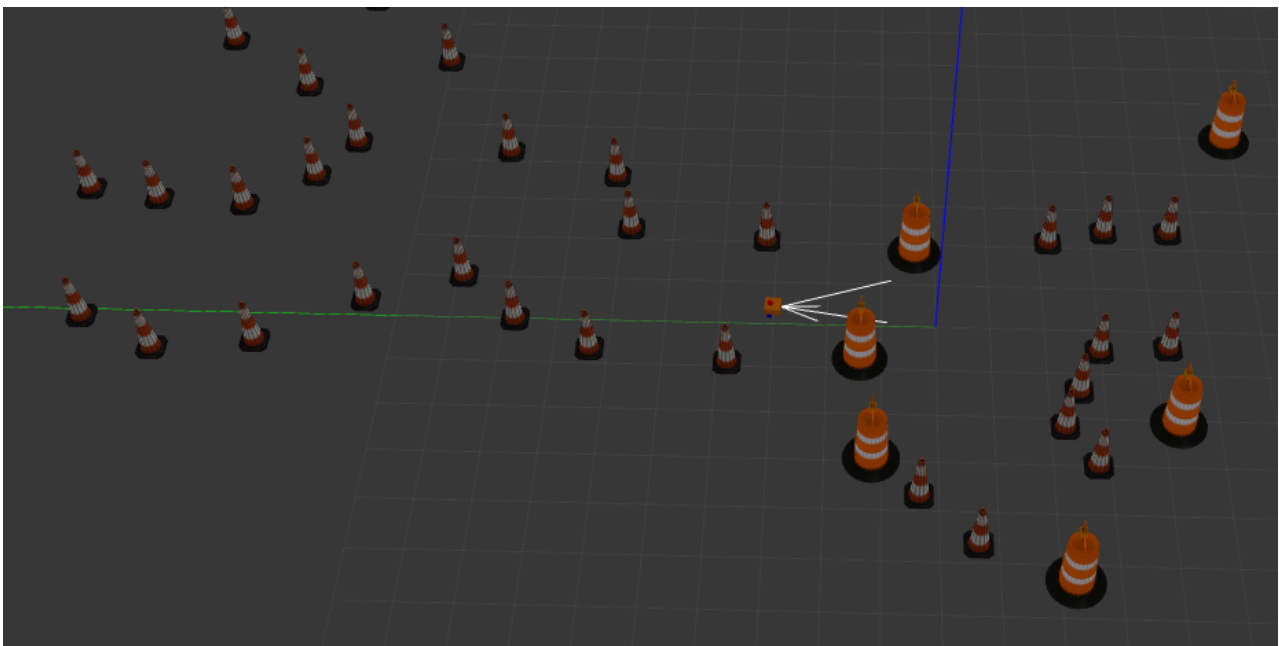
- **Process:** As the rover is manually driven (teleoperated) through the Gazebo world, slam_toolbox processes the incoming 2D LiDAR scans and tf (transform) data.
- **Output:** It generates a 2D occupancy grid map (nav_msgs/OccupancyGrid) in real-time. The package performs graph-based SLAM, enabling loop closure and map correction.
- **Map Saving:** Once the environment is fully explored, the final map is saved to disk using the map_saver utility.

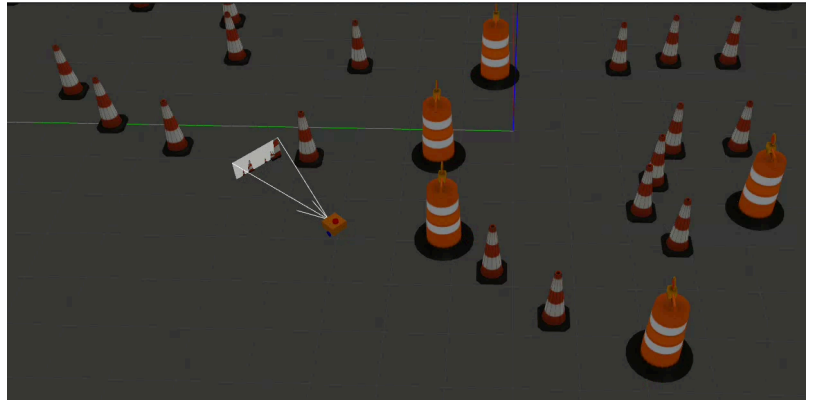
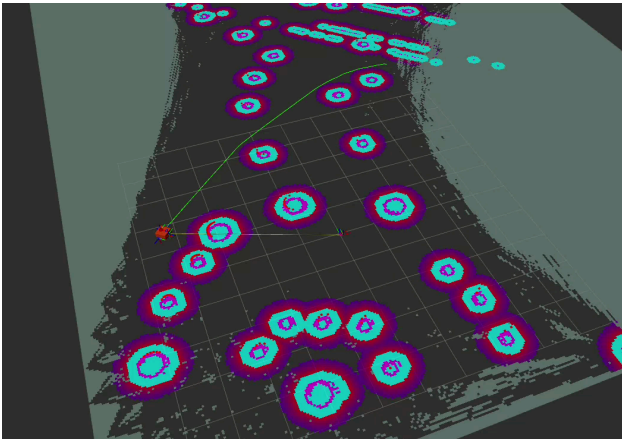
6.2.6 2D Localization and Navigation (Nav2)

With the static map loaded, the system transitions to the Nav2 stack for autonomous navigation.

- **Localization:** The AMCL (Adaptive Monte Carlo Localization) node is used. It estimates the rover's pose within the saved map by matching current LiDAR scans against the map.
- **Path Planning:** Nav2's planner (e.g., Smac Planner) generates an optimal global path from the rover's current location to a specified goal.
- **Dynamic Obstacle Avoidance:** The Nav2 controller and costmap layers (e.g., Obstacle Layer) are configured to handle dynamic objects. Obstacles detected by the LiDAR that are not part of the static map are added to the local costmap, forcing the controller to compute a new, safe local path.

Lunar rover:





simulation results:

 robot_simulation.webm

 Accurate Robot Navigation

6.2.7 Limitations and Future Work: 3D VSLAM for Lunar Terrain

The current implementation successfully navigates a "plane surface" but is fundamentally unequipped for a realistic lunar environment, which is characterized by craters, rocks, and slopes.

The next phase of this project is to transition from 2D SLAM to 3D mapping and navigation:

- **3D LiDAR Integration:** A 3D LiDAR (e.g., Velodyne) will be added to the model to generate 3D point clouds (sensor_msgs/PointCloud2), capturing the true 3D structure of the terrain.
- **VSLAM Integration:** Visual SLAM (VSLAM) algorithms (such as ORB-SLAM3 or DROID-SLAM) will be implemented to use the onboard cameras. This provides robust motion estimation and dense 3D reconstruction, which is critical for traversing complex terrain.
- **Sensor Fusion:** The ultimate goal is to fuse data from the 3D LiDAR, VSLAM cameras, and IMU (e.g., using rtabmap_ros or a custom EKF/graph-based filter) to build a 3D map (such as an OctoMap) and enable true 3D navigation on the uneven lunar surface.

Future Lunar rover:



6.3 Data Collection and VSLAM Validation (GITAM Autonomous Vehicle (GAV) Platform Buggy)

6.3.1 Overview

This section details the methodology for collecting high-fidelity, multi-modal sensor datasets using the GITAM Autonomous Vehicle (GAV) autonomous buggy (as described in Section 5). The platform's primary function in this research is to serve as a sophisticated data collection testbed.

The system's operational navigation, based on a LiDAR-Centric SLAM approach, provides a highly accurate, centimeter-level ground-truth pose. This ground-truth trajectory is recorded in complete synchronization with all visual and inertial sensors like IntelRealSense D435i, Xsens IMU.

The resulting ROS bag files create a rich, large-scale dataset. This dataset is then used for the offline development, rigorous testing, and quantitative validation of new Visual Odometry (VO) and Visual SLAM (VSLAM) algorithms, such as DROID-SLAM and ORB-SLAM.

6.3.2 Physical Testbed and Platform Model

Unlike the simulated rover, this platform is a real-world vehicle operating in complex environments (e.g., industrial campuses).

- **Vehicle Platform:** The GITAM Autonomous Vehicle (GAV) Buggy provides a highly stable sensor platform. Its precise mechanical specifications (e.g., 4072 mm length) and known kinematic model are essential for accurate ground-truth modeling.
- **Operating Environment:** Data is collected in real-world scenarios, including varied lighting conditions, dynamic obstacles (people, other vehicles), and diverse terrain features, providing a challenging and realistic benchmark for VSLAM.

6.3.3 Sensor Configuration and Data Synchronization

The platform is equipped with a redundant, high-end sensor suite. For the purpose of VSLAM development, the sensors are leveraged as follows:

VSLAM Development Inputs:

- **Intel RealSense D435i:** Provides the primary visual data streams (Stereo, RGB-D) and a secondary IMU feed.

Ground-Truth Generation:

- **Velodyne VLP-32C (3D LiDAR):** Used to generate the persistent 3D map and provide the primary, high-accuracy 6-DoF localization pose.
- **Xsens MTI-610 (IMU):** Provides high-frequency, low-drift inertial data, which is fused into the ground-truth estimate.
- **Autonics Wheel Encoder:** Offers a non-slip odometry baseline for velocity and distance validation.

All sensor streams are managed, time-stamped, and recorded into a unified ROS (Robot Operating System) data bus, ensuring precise time synchronization across all modalities.

6.3.4 Ground-Truth Data Generation (LiDAR-SLAM)

The ground-truth trajectory is not simulated but generated by the vehicle's robust LiDAR-based navigation stack (as described in 5.5.2).

- **Process:** While the buggy is manually or autonomously driven, the LiDAR-SLAM algorithm continuously matches live VLP-32C point clouds against a pre-built 3D persistent map.

- **Output:** This process generates a continuous, highly accurate 6-DoF pose estimate (tf and geometry_msgs/PoseStamped). This data, combined with the wheel encoder odometry, is recorded as the ground-truth trajectory.

6.3.5 Future Work: VSLAM Validation Workflow and System Integration

The foundational data collection enables the primary future work: validating and integrating a VSLAM system. This work will follow a structured validation workflow before real-time integration.

6.3.5.1 Offline Processing and Benchmarking

The core next step is to use the collected ROS bags for rigorous offline validation.

- **Replay:** The bags will be "replayed" (`roslaunch rosbag play`) on a development machine.
- **Processing:** The VSLAM algorithm under test (e.g., DROID-SLAM, ORB-SLAM3) will be launched, subscribing only to the visual data (RealSense images/IMU) from the bag file.
- **Trajectory Generation:** The VSLAM algorithm will process the visual data and compute its own estimated trajectory, unaware of the ground truth.
- **Quantitative Validation:** The VSLAM-generated trajectory will be compared directly against the ground-truth trajectory (from the LiDAR-SLAM) saved within the same ROS bag. This allows for precise, metric-level error analysis using Absolute Trajectory Error (ATE) and Relative Pose Error (RPE).

6.3.5.2 Real-Time Deployment and Optimization

Once an algorithm is validated offline, it will be deployed onto the buggy's ThinkEdge SE50 industrial computer for live testing. This phase will require significant code optimization to achieve real-time performance, including leveraging the GPU by porting algorithms to CUDA or using inference accelerators like TensorRT for deep-learning-based components.

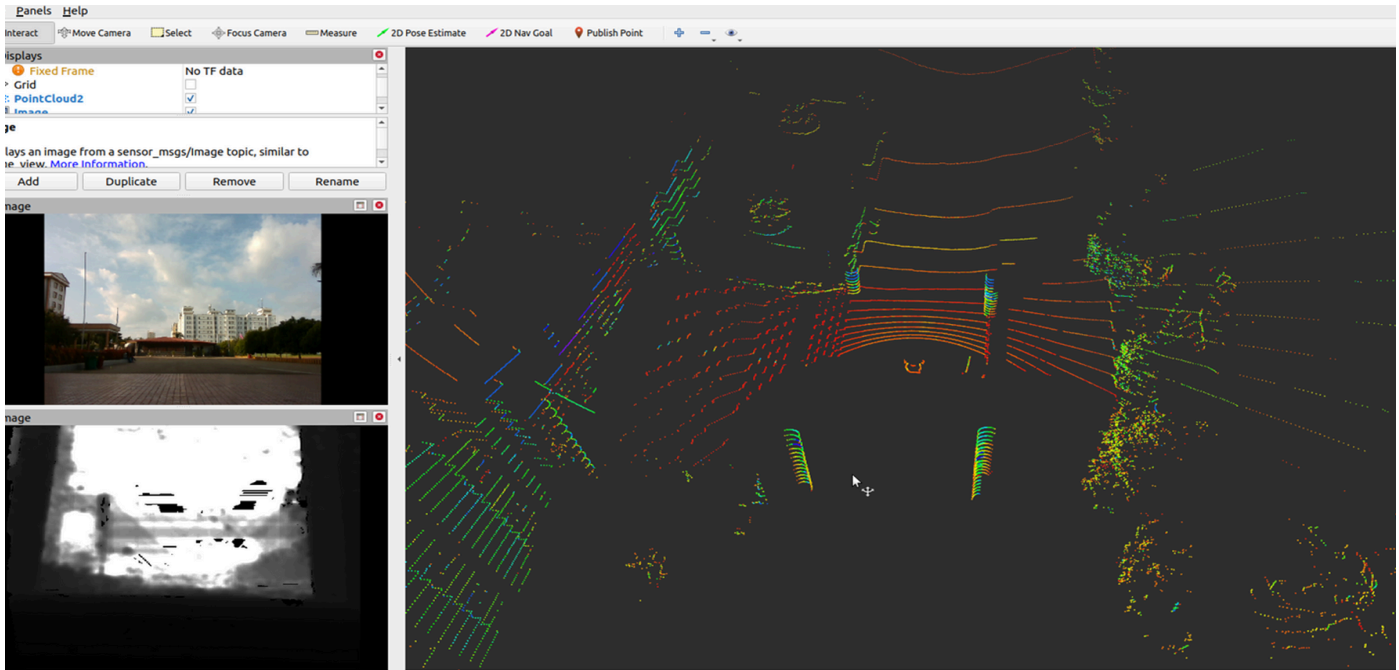
6.3.5.3 Robust Sensor Fusion (VSLAM + LiDAR-SLAM)

The ultimate objective is to create a resilient, all-weather navigation system by fusing the VSLAM pose estimates with the primary LiDAR-SLAM and Xsens IMU data. This will be implemented using an advanced filter-based (e.g., Extended Kalman Filter - EKF) or graph-based optimization backend. This fused system will be redundant, allowing the buggy to maintain accurate localization even if one modality fails.

6.3.5.4 Advanced Visual Perception and Navigation

With a validated real-time visual pipeline, future work will also integrate visual object detection (e.g., YOLO-based models) and semantic segmentation. This will allow the navigation stack to not only detect an obstacle but also classify it (e.g., pedestrian, vehicle, curb), leading to more intelligent and context-aware path planning.

Buggy images and rviz images:



Real-time sensor data from Buggy

7. Conclusion

7.1. Summary of Key Findings and Contributions of the Report

This report has provided a comprehensive survey of Visual Odometry (VO) and Visual SLAM (VSLAM), beginning with fundamental principles and concluding with practical implementation and validation.

A key finding of the state-of-the-art survey is the clear technological evolution from established, feature-based geometric methods like **ORB-SLAM2** to next-generation, end-to-end deep learning architectures like **DROID-SLAM**. While classic methods provide versatility and a robust Bundle Adjustment backend, learning-based approaches demonstrate a monumental leap in tracking robustness and accuracy, largely by integrating geometric principles (like Differentiable Bundle Adjustment) directly into the network architecture.

The technical survey in Chapter 4 underscored a critical, practical finding: the performance of any VSLAM system is fundamentally dependent on meticulous **sensor calibration and synchronization**. The report detailed the necessity of precise intrinsic (camera, IMU) and extrinsic (camera-to-IMU) calibration, and emphasized that robust, high-performance Visual-Inertial Odometry (VIO) is only feasible with hardware-level time synchronization to eliminate timing jitter.

The primary contributions of this report are centered on the development of a real-world validation framework:

1. **Platform Development:** The report detailed the design and sensor configuration of the **GITAM Autonomous Vehicle (GAV) autonomous buggy**, a sophisticated, multi-sensor data collection platform.
2. **Ground-Truth Data Generation:** It established the platform's methodology, wherein its high-accuracy, LiDAR-based SLAM and Xsens IMU serve to generate a reliable, centimeter-level ground-truth trajectory.
3. **Dataset Collection:** The platform was successfully used to collect a novel, large-scale, multi-modal dataset

(synchronized stereo, RGB-D, IMU, and LiDAR ground-truth) in ROS bag format, which serves as the core asset for all future validation.

4. **Implementation and Analysis:** The report presented the successful implementation and analysis of DROID-SLAM, demonstrating its 3D reconstruction capabilities. Furthermore, it detailed the development of a 2D SLAM and navigation stack for a simulated Lunar Rover, which establishes a baseline for future 3D navigation research.
5. **Validation Workflow:** Finally, the report defined a clear and rigorous workflow (Section 6.3.5) for using the collected dataset to quantitatively benchmark VSLAM algorithms (e.g., ATE/RPE analysis) against the high-fidelity ground truth.

7.2. Future Work and Research Outlook

This report's findings and contributions lay the groundwork for a clear and ambitious research path. The future work is divided between the immediate validation on the physical platform and the parallel development of the 3D simulation.

The most immediate priority is the **offline validation and benchmarking** described in Section 6.3.5.1. This involves replaying the collected ROS bags and processing the visual-inertial data through VSLAM algorithms. This benchmarking is not just for selection, but is the foundational step for **improving the VSLAM algorithms themselves**. This includes deep algorithm refinement, such as fine-tuning DROID-SLAM's network parameters, optimizing front-end feature tracking, and critically, **re-training deep learning models** on our platform-specific dataset to maximize domain adaptation and robustness to our specific environmental conditions.

Following successful offline validation, the next steps for the **GITAM Autonomous Vehicle (GAV) Buggy platform** are:

1. **Real-Time Deployment:** To port and optimize the chosen VSLAM algorithm (e.g., using CUDA or TensorRT) to run in real-time on the buggy's ThinkEdge SE50 industrial computer. This will involve creating optimized ROS 2 nodes and managing computational resources to meet the 15-30 FPS target required for stable tracking.
2. **Robust Sensor Fusion Integration:** To achieve the "ultimate objective" of an all-weather, resilient navigation system by tightly fusing the real-time VSLAM pose estimate with the existing LiDAR-SLAM and Xsens IMU data. This is a significant challenge that involves developing an advanced filter or graph-based optimization backend. This backend must correctly manage data of varying frequencies (100Hz IMU vs. 30Hz VSLAM vs. 10Hz LiDAR-SLAM), handle potential sensor dropouts, and properly weight the covariance of each estimate to create a single, stable, and highly redundant pose for the navigation stack.
3. **Advanced Visual Perception:** To integrate modern object detection (e.g., YOLO) and semantic segmentation models with the VSLAM pipeline. This will enable the navigation stack to move beyond simple obstacle detection and perform context-aware path planning by classifying and predicting the behavior of pedestrians, vehicles, and other dynamic agents.

Concurrently, development on the **Lunar Rover simulation** (Section 6.2.7) will focus on **ROS simulation deployment and 3D navigation**. This involves:

1. **3D Environment Modeling:** Upgrading the Gazebo simulation from a 2D-planar world to a realistic 3D lunar environment with craters, rocks, and slopes to properly test 3D motion.
2. **Full ROS 2 Integration Testing:** This involves deploying the complete VSLAM and LiDAR SLAM algorithms (e.g., DROID-SLAM, LIO-SAM) as nodes within the ROS 2 simulation. This provides a safe, controlled environment to debug the full software stack, including data flow through ROS topics, `tf` tree management, and node lifecycle.
3. **3D Navigation Stack Validation:** Building and validating a complete 3D navigation stack that *consumes* the VSLAM output. This simulated validation is crucial for de-risking the complex algorithm integration and 3D path planning logic before attempting real-world deployment on hardware.

In the broader research outlook, these efforts align with major industry trends toward multi-agent collaborative SLAM (fleet management) and semantic SLAM, where systems build maps that are not just geometric but also rich in contextual understanding.

Bengaluru City Office No 5/1, First Floor, Prestige Terraces, Union Street, Infantry Road, Bengaluru - 560003 Karnataka, India



GITAM
GLOBAL INSTITUTE OF TECHNOLOGY AND MANAGEMENT



Bengaluru



